Topic Title: Wi-Fi HaLow-Based Mesh Network Routing for Underground Mine Sites

Student Name: Hongtao Liu **Student ID: z5500132**

A. **Problem statement**

In a wireless mesh network, each node can act as a router or repeater. Nodes forward data for each other and create multiple possible paths. This is very useful in mines. However, the network topology changes often because some devices move. In this case, traditional static routing protocols do not work well. There are several challenges: node mobility, limited mem ory, limited energy, and unstable links. To handle these problems and avoid overloading a single path, we need a well-designed routing algorithm. The key question is: can we design a newrouting method that still provides stable, high-quality communication under such changing conditions?

B. **Objective**

Find the optimal network route in underground mine sites
Find the relationship of the AP's physical location in protocol

C. **My solution**

Design a location-aware routing method
Each node uses its (estimated) position and RSSI to choose better next hops
Use deep learning to improve the routing planning

D. **Contributions** (at most one per line, most important first)

Scenario-driven communication stack for mines.
Experimental characterisation of Wi-Fi HaLow in corridor / mine-like environments.
Calibrated NS-3 of the corridor and tunnel path-loss model.
Position-aware routing and RL-based path planning.
Future-oriented framework for robust mine networking.

E. **Suggestions for future work**

Measurement-to-model calibration aligns NS-3 link behavior with field reality (walls, corners), fixing "paper-shortest $\neq$ usable route".
Localization-to-routing coupling (RMSE-checked positions fed into route selection) improves delivery rate under corridor constraints.
Robustness & safety: Adversarial training with link/node outages and interference spikes; track recovery time/service continuity.

While I may have benefited from discussion with other people, I certify that this report is entirely my own work, except where appropriately documented acknowledgements are included.

Signature: Hongtao Liu Date: 20/ nov / 2025

Pointers

List relevant page numbers in the column on the left. Be precise and selective: Don't list all pages of your report!

P8	Problem Statement
P9	Objective

Theory (up to 5 most relevant ideas)

P18	Obtain RSS data through simulation
P19	Channel and PHY Error Model
P22	DQN learning

Method of solution (up to 5 most relevant points)

P29	Real-world channel measurement and calibration
P24	RSSI-based localisation and positioning validation
P37	NS-3 "digital twin" of the corridor / tunnel
P43	Reinforcement-learning–based routing environment
P47	Dueling DQN training and path optimisation

Contributions (most important first)

P24	Scenario-driven communication stack for mines.
P29	Experimental characterisation of Wi-Fi HaLow in mine-like environments.
P37	Calibrated NS-3 of the corridor and tunnel path-loss model.
P43	Position-aware routing and RL-based path planning.
P52	Future-oriented framework for robust mine networking.

My work

	System block diagrams/algorithms/equations solved
	Description of assessment criteria used
	Description of procedure (e.g. for experiments)

Results

P30	Experiment measurement results
P40	NS3 simulation result
P49	Results of Training curves and path visalization

Conclusion

P52	Statement of whether the outcomes met the objectives
P52	Suggestions for future research

Literature: (up to 5 most important references)

P13	E. Alotaibi and B. Mukherjee
P15	S.Bari, F. Anwar, andM.H.Masud
P15	Q. Li, Z. Huo, L. Wen et al.
P16	H. Xujian and W. Hao
P16	J. Feng, Z. Liu, and Y. Ji

SCHOOL OF ELECTRICAL ENGINEERING
AND TELECOMMUNICATIONS

Wi-Fi HaLow-Based Mesh Network Routing for Underground Mine Sites

by

Hongtao Liu

Student ID: z5500132

Thesis submitted as a requirement for the degree

Master of Engineering (Electrical Engineering)

Submitted: November 26, 2025
Supervisor: Binghao Li and Xiaoyu Ai

Abstract

Australia's mining industry is in a developing progress driven by the Internet of Things (IoT). This paper talks about Wi-Fi HaLow for its long-range, connectivity in challenging underground environments, conducting a comparative analysis with LoRa. It examines the tunnel-based mesh networks typical of mines. A real world test was done to get the true parameters of Wi-Fi Halow. A simulation validates the accuracy of Wi-Fi HaLow-based positioning. Furthermore, strategic integration of machine learning is introduced to determine and optimize the path planning of access points (APs). The paper concludes with plans for a comprehensive system design and outlines directions for future enhancement.

Acknowledgements

I extend my thanks to Binghao Li and XiaoYu Ai for their valuable insights and assistance. I am also deeply grateful to my supervisors for their guidance in shaping the direction of this research, and to Professor Aruna for her thoughtful questions and feedback during the presentation.

Abbreviations

AID Association Identifier

AODV Ad hoc On-Demand Distance Vector

AOMDV Ad hoc On-Demand Multipath Distance Vector

AP Access Point

BER Bit Error Rate

BPSK Binary Phase-Shift Keying

DQN Deep Q-Network

DSDV Destination-Sequenced Distance Vector

FSK Frequency-Shift Keying

GAF Geographic Adaptive Fidelity

GI Guard Interval

GPS Global Positioning System

HWMP Hybrid Wireless Mesh Protocol

ICMP Internet Control Message Protocol

IEEE Institute of Electrical and Electronics Engineers

IoT Internet of Things

KNN K-Nearest Neighbors

LDPC Low-Density Parity-Check

LoRa Long Range Radio

LOS Line-of-Sight

MAC Medium Access Control

MANET Mobile Ad hoc Network

MCS Modulation and Coding Scheme

ML Machine Learning

MIMO Multiple-Input Multiple-Output

NLOS Non-Line-of-Sight

NS-3 Network Simulator 3

PER Packet Error Rate

PHY Physical Layer

PL Path Loss

QoS Quality of Service

QAM Quadrature Amplitude Modulation

RAW Restricted Access Window

RSS Received Signal Strength

RSSI Received Signal Strength Indicator

RTT Round-Trip Time

RMSE Root Mean Square Error

SINR Signal-to-Interference-plus-Noise Ratio

SNR Signal-to-Noise Ratio

STA Station

TD Temporal-Difference

TIM Traffic Indication Map

UDP User Datagram Protocol

UAV Unmanned Aerial Vehicle

WLAN Wireless Local Area Network

WMN Wireless Mesh Network

WSN Wireless Sensor Network

Contents

Acknowledgements **1**

Abbreviations **2**

Contents **5**

1 Introduction **7**

1.1 Background 7

1.2 Problem Statement 8

1.3 Solution 9

1.4 Chapter Outline 9

2 Literature Review **10**

2.1 Features of Wi-Fi HaLow 10

2.2 Mine-site wireless mesh network 11

2.3 Traditional Algorithms of Wireless Mesh Network 12

2.3.1 Geographical RA 13

2.3.2 Hybrid RA 15

2.4 Using Wi-Fi to locate 15

2.4.1 Received signal strength 16

3 Theory **18**

3.1 Obtain RSS data through simulation 18

3.2 Channel and PHY Error Model 19

3.2.1 Path Loss and Received Power 20

3.2.2 Noise Power and SNR 20

3.2.3 Propagation Delay 20

3.2.4 MCS-Dependent Error Model (YansErrorRate) 20
3.2.5 Topology for Reproducibility 21
3.3 DQN learning 22

4 Experiment and simulation **24**
4.1 Test the accuracy of Wi-Fi Halow location 24
4.2 Experiment 29
4.2.1 RSSI measurement preparation in hardware 29
4.2.2 Test environment 30
4.2.3 Measurement results and discussion 30
4.3 NS3 simulation 37
4.3.1 Build NS3 environment 37
4.3.2 NS3 environment test 38
4.3.3 Build WiFi mesh network 39
4.3.4 The result of simulation 40

5 Optimal mesh network path Planning **43**
5.1 Design of the code 43
5.1.1 Reinforcement–Learning-Based Path Planning Implementation 43
5.1.2 Mine Tunnel Environment 43
5.1.3 Channel and Path-Loss Model 44
5.1.4 State Representation 45
5.1.5 Action Space and STA Motion Model 46
5.1.6 Reward Design 46
5.2 Dueling DQN Architecture 47
5.2.1 DQN Agent and Training Procedure 48
5.2.2 Results of Training Curves and Path Visualization 49

6 Conclusion **52**

Bibliography **53**

Chapter 1

Introduction

1.1 Background

In underground mines, the Internet of Things (IoT) links sensors, wireless communication, and control systems. These devices form a monitoring and control network. It can track the environment in real time, support automatic decision-making, and enable remote operation.

A typical example is Rio Tinto's "Smart Mine". In this project, a large number of sensors are deployed across the mine. Autonomous haul trucks and intelligent monitoring platforms are also used. Together, they increase production efficiency and improve worker safety. [1]

Figure 1.1: IoT application in mine industry

An underground mine is a complex engineering system. It consists of many components arranged in three-dimensional space. Teamwork is essential to keep extraction safe, efficient, and sustainable.

The infrastructure includes production parts such as shafts, drifts, and stopes. It is also supported by transport systems, ventilation, and drainage.[2] All of these elements must be surrounded by enough rock. This rock cover keeps the main access roads and auxiliary facilities stable. The overall layout follows a clear spatial hierarchy. The main shaft is usually placed on one side of the ore body. It connects the surface to the main haulage level and the stopes. Stopes are set in several vertical levels inside the ore body. Between stope levels, rock layers of 10–40,m are left to avoid interaction. On each level, main haulage drifts and ventilation drifts run horizontally. Crosscuts link these drifts to each stope. A key design rule is to keep 5–10,m of rock around all drifts. This helps maintain structural safety and stable airflow. In addition, at least 30,m of overburden rock is kept above the highest workings. This reduces the risk of ground movement and surface subsidence.

The complexity of the underground environment necessitates accounting for more variables than on the surface, including obstacles in the pathway.[3]. Designed for IoT applications, Wi-Fi HaLow (IEEE 802.11ah) supports communications over distances of up to one kilometer, thereby decreasing node density and easing deployment. Its strong penetration capability through solid materials like walls and metals further enhances usability. However, a number of technical challenges must still be addressed in mesh network configurations.

1.2 Problem Statement

In a wireless mesh network, each node can act as a router or repeater. Nodes forward data for each other and create multiple possible paths. This is very useful in mines. However, the network topology changes often because some devices move. In this case, traditional static routing protocols do not work well. There are several challenges: node mobility, limited memory, limited energy, and unstable links. To handle these problems and avoid overloading a single path, we need a well-designed routing algorithm. The key question is: can we design a new routing method that still provides stable, high-quality communication under such changing conditions?

1.3 Solution

In this work, I use physical location information to help design the routing scheme. First, I measure Wi-Fi HaLow signal strength and link quality in a real corridor and near mine-like environments. These tests give a basic map of how the signal changes with distance and obstacles. Next, I build a simulation model in NS-3 based on these measurements. The model reproduces the tunnel layout and the main channel features. On top of this model, I design a location-aware routing method. Each node uses its (estimated) position and RSSI to choose better next hops, so that blind flooding is reduced and the number of relay hops is kept low. Finally, I apply machine learning to further optimise the route selection. Physical location, link quality, and traffic load are used as input features. The trained model helps pick more stable and efficient paths under changing network conditions.

1.4 Chapter Outline

Chapter 2 explains the background for this document. Chapter 3 presents the theory of the total project. Chapter 4 shows the experiment and simulation in the real environment and NS3. Chapter 5 explains the steps of codes and shows the result of the optimal network route planning .

Chapter 2

Literature Review

2.1 Features of Wi-Fi HaLow

Wi-Fi HaLow (IEEE 802.11ah) is a wireless communication standard specifically developed for Internet of Things (IoT) applications. Operating primarily in the sub-1 GHz frequency band (below 900 MHz), Wi-Fi HaLow leverages its lower frequency to deliver several key advantages for mining environments. It enables long-range communication of up to one kilometer, thereby reducing the number of required nodes and simplifying deployment. Furthermore, the low-frequency signal exhibits superior penetration through obstacles such as walls, rock formations, and metallic structures, making it highly suitable for structurally complex underground settings.

In the context of wireless mesh networks, Wi-Fi HaLow offers additional benefits. It achieves a data rate of up to 86.7 Mbps in the sub-1 GHz band, significantly outperforming alternatives such as Z-Wave, NB-IoT, and FSK-based systems. With a transmission range of approximately 1 km, it remains competitive with other long-range WLAN technologies. Moreover, Wi-Fi HaLow supports large-scale node connectivity by extending the Association Identifier (AID) range from 2,007 to 8,191. This allows each access point to theoretically accommodate up to 8,191 devices and enables direct internet access without the need for a dedicated gateway.[4]. Wi-Fi HaLow's low power consumption is a key factor in its advantage over other Wi-Fi technologies. Furthermore, its native support for IP-based networks facilitates seamless customer access to cloud applications, while eliminating the need for additional infrastructure, thereby significantly reducing the total cost of ownership.

When selecting low-power wireless communication technologies for underground environ-

ments, Wi-Fi HaLow and LoRa are two prominent options. LoRa is renowned for its exceptional range and minimal power consumption, making it well-suited for applications involving intermittent, low-volume sensor data. However, its low data rates restrict its use to small-packet transmissions at slow speeds. In contrast, Wi-Fi HaLow offers a compelling balance of range, power efficiency, and substantially higher throughput, supporting more data-intensive operations without sacrificing coverage.Comparative Analysis of LoRa and Wi-Fi HaLow is shown in the table 2.1.

Table 2.1: Comparative Analysis of LoRa and Wi-Fi HaLow

	Wi-Fi HaLow	LoRa[5]
Operating band	Sub-GHz ISM, around 900 MHz	Sub-GHz ISM, typically 433/868/915 MHz
Typical distance	About 1–2 km in outdoor links	Up to roughly 10–15 km for low-rate links
Data rate	Approx. 150 Kbps–346 Mbps (mode dependent)	Approx. 0.3 Kbps–50 Kbps
Power consumption	Low-power operation (higher than LoRa but still efficient)	Very low power consumption
Network topology	Wi-Fi infrastructure with IP-based AP–STA connections	Gateway-centred star / star-of-stars

Operating in the sub-1,GHz band, similar to LoRa, Wi-Fi HaLow can provide much higher data rates. These rates are enough to carry video, voice, and large volumes of data. This makes Wi-Fi HaLow well suited to underground mines, where many real-time devices work together and need reliable, high-capacity links.

2.2 Mine-site wireless mesh network

A wireless mesh network is a distributed, multi-hop communication system. It offers high capacity and high data rates, which makes it different from many traditional network designs.[6] In mining communication, node mobility is usually very low. This matches the strengths of mesh networks and helps improve overall transmission efficiency.

To provide full wireless coverage along mine roadways, a backbone is built with wireless mesh routers.[7] These routers offer broadband access for underground terminals. One router

is linked directly to the surface wired network and acts as a gateway. It gathers data from underground applications and sends it to the emergency command and dispatch center on the surface.

Wireless Mesh Networks (WMNs) evolved from Mobile Ad Hoc Networks (MANETs). They reuse many routing ideas, such as source routing. However, underground coal mine networks have special features. Most nodes are fixed or move very slowly. They are often arranged in a line or chain along the roadway.

The main strength of WMNs lies in their routing protocols. Network performance strongly depends on how well these protocols are designed. In underground coal mines, nodes are powered by cables. So, energy consumption is not the main limitation for routing. Instead, the key goal is to balance the load among multiple gateways.

In tunnel environments, the tunnel shape has a strong impact on wireless propagation. Metro and highway tunnels usually have smooth, continuous walls. These walls act like multimode waveguides. Radio waves reflect many times and mainly travel along the tunnel axis. This reduces path loss and extends the communication range.

Mine tunnels are different. Their walls are rough and irregular. There are also supports, cables, and machines. These objects scatter and absorb the signal. As a result, path loss is higher and links break more easily, especially at 5.8,GHz.

Measurements show that the 2.4,GHz band suits complex mine tunnels better.[8] Its longer wavelength improves diffraction around obstacles. It produces lower path loss, fewer interruptions, and more stable delay spread and coherence bandwidth. These findings are important for designing robust tunnel communication systems.

In regular tunnels, such as metro tunnels, higher frequency bands can be used to raise data rates. In complex mine tunnels, lower frequency bands should be chosen first. They can be combined with directional antennas or relay nodes to improve link reliability. In the future, adding techniques such as millimetre wave links, MIMO, and adaptive algorithms may further enhance wireless performance in different tunnel scenarios.

2.3 Traditional Algorithms of Wireless Mesh Network

The traditional classification of routing algorithms, based on criteria such as proactive versus reactive or centralized versus distributed, has evolved into a more nuanced framework that

categorizes algorithms according to their core technological mechanisms.The graph of classification is show in the figure 2.1.

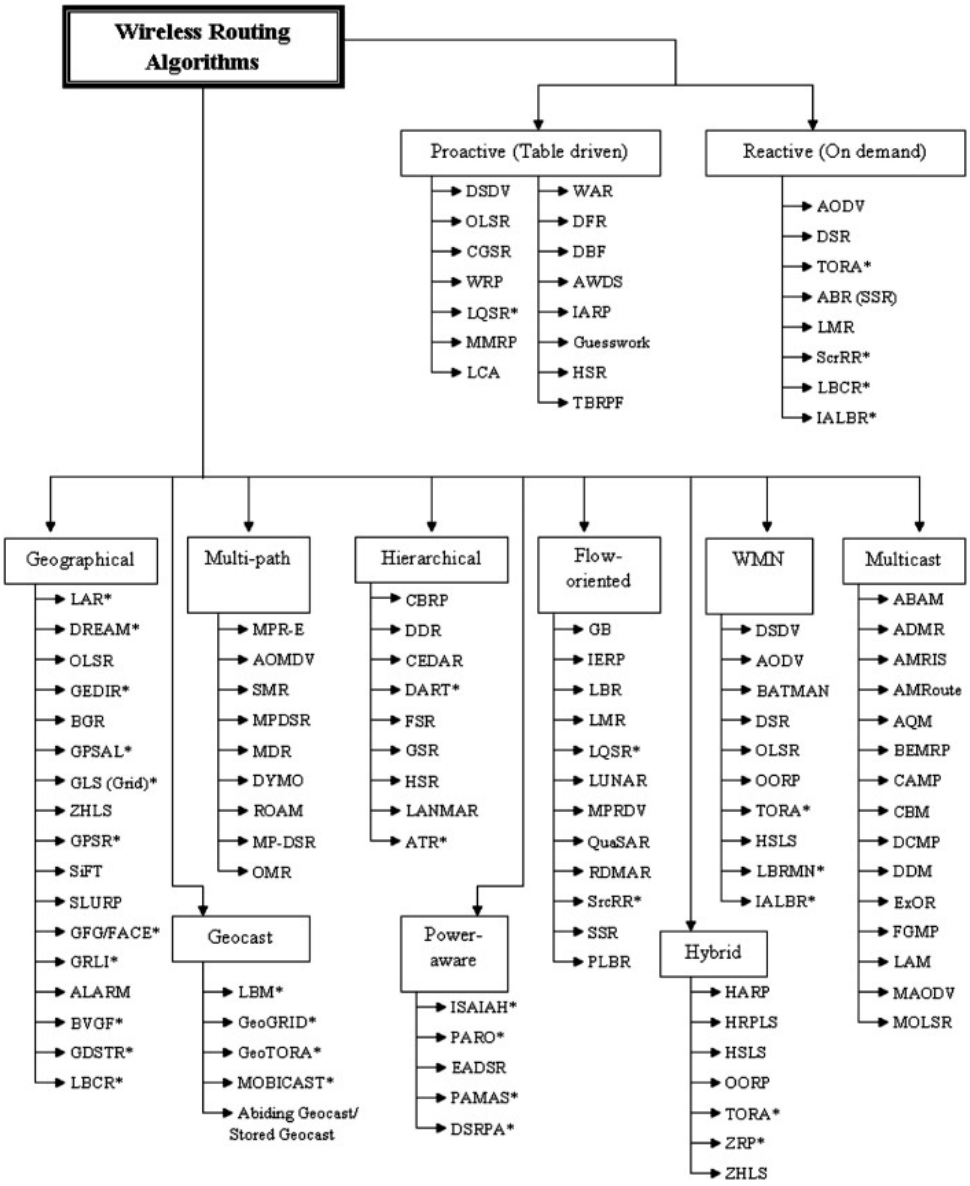

Figure 2.1: Classification of RA

2.3.1 Geographical RA

A foundational advancement in modern communication technology has been the development of the Global Positioning System (GPS). It provides not only precise global positioning capabilities but also delivers universal time synchronization, forming a critical infrastructure for networked devices worldwide.[9]. The development of this capability has established a powerful routing paradigm known as Geographical Routing (or Position-Based Routing).

Unlike traditional methods that depend heavily on IP addresses or global topology knowledge, geographical routing uses the physical coordinates of the destination node to steer packet forwarding. A transmitting node selects the next hop based on the geographical positions of its neighbors relative to the final destination, greatly simplifying routing logic and cutting protocol overhead.

This model relies on two fundamental premises:

Each node can determine its own location, generally via GPS or other localization systems.

The source node knows the destination's geographic position. This position can come from a location service or from periodic updates.

Using this information, routing protocols such as Greedy Forwarding, Face Routing and Geographic Adaptive Fidelity (GAF) can react to changes in the network. They are well suited to MANETs, VANETs and WSNs, where the topology changes often and there is no central controller.

Geographic routing also scales well. It saves energy because it does not require a global routing table at every node. This makes it a strong candidate for future wireless systems.

In general, position information can improve routing performance in wireless networks. Localization algorithms help cut control overhead. Most geographic routing schemes are naturally scalable and tolerant to node failures. However, real-time position data may be noisy or outdated. In practice, not every node can always know its own coordinates. So, the final network performance depends on a careful design. We must balance how nodes are spread in space and avoid traffic hotspots. The performance gain from the geographic route algorithm (RA) is shown in Fig. 2.2.

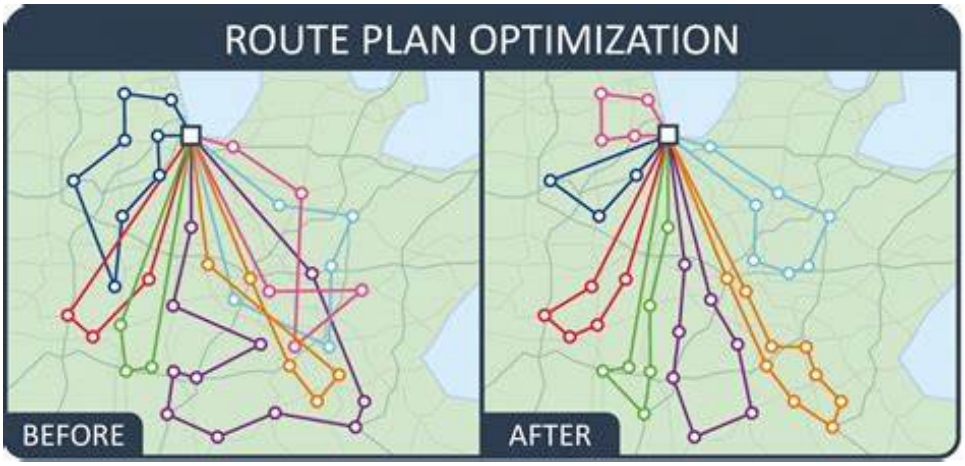

Figure 2.2: Improvement of geographical RA

2.3.2 Hybrid RA

The hybrid routing protocol combines ideas from proactive and reactive schemes. When the destination node is nearby, it uses a proactive method. Control messages are limited to a small area, and routing tables are kept up to date with low cost. When the destination is far away, it switches to an on-demand mode. Routes are discovered only when needed and routing requests are sent reactively. Typical hybrid protocols include the Hybrid Wireless Mesh Protocol (HWMP) and the Zone Routing Protocol (ZRP)[10]. In [11], HWMP is chosen as the default routing protocol. The reason is that many traditional protocols depend on the IP layer for multi-hop forwarding and cannot express wireless link quality well. HWMP instead uses a link-state framework and adopts the airtime metric as the default cost. This metric reflects the channel resources consumed when a frame is sent over a wireless link. In the multipath version of HWMP, the main route is not simply the first path that is found. Among all candidate paths, the protocol picks the one with the smallest total cost as the primary route. The goal of the multipath discovery process is to quickly obtain a near-optimal path between the source and the destination[12]. Li's work[13] considers the roadway layout and the Minimum Ignition Energy (MIE) safety requirement. He analyses electromagnetic wave attenuation and designs safe spacing between nodes. On this basis, he combines proactive and reactive routing to form a hybrid scheme suited to harsh underground mine conditions. The hybrid protocol keeps the flexibility of on-demand routing and the low delay of proactive routing. In that study, the Mesh network is connected to a command and dispatch platform to build an emergency rescue system. The system can monitor underground environmental parameters in real time and also supports reliable multimedia communication during rescue and dispatch operations.

2.4 Using Wi-Fi to locate

The absence of GPS signals in underground mines makes Wi-Fi HaLow access points a viable alternative for node localization. A primary challenge in such Wi-Fi-based positioning systems, however, lies in mitigating the impact of signal fluctuations on positioning accuracy.[14]. Due to the significant impact of environmental factors—such as multipath effects, signal attenuation, and noise interference—on Wi-Fi signals, fingerprint-based positioning methods demonstrate higher accuracy compared to traditional Wi-Fi positioning algorithms.

Each physical location is associated with a distinct fingerprint, which may be represented

by either single-dimensional or multi-dimensional feature vectors.[15]. For example, when a target device is transmitting or receiving information, its signal fingerprint can be composed of one or more signal or information characteristics—with signal strength being the most commonly used feature. If the target device is actively transmitting signals, multiple fixed receiving devices can detect and measure these signals, enabling the positioning of the target. [16].

2.4.1 Received signal strength

The Received Signal Strength (RSS) at a wireless device is inherently dependent on its geographical location. RSS measurement is straightforward to implement, as it is already a fundamental parameter required for the normal operation of most wireless communication systems. In practice, RSS data is widely used for tasks such as link quality assessment, handover decisions, and transmission rate adaptation.[17].Received Signal Strength (RSS) is independent of signal bandwidth and does not require high-bandwidth channels, as the signals of most common communication systems operate within relatively narrow bands. These characteristics make RSS a widely adopted signal feature for positioning applications.

Assume a fixed signal transmission source. The attenuation of the average Received Signal Strength (RSS), measured in decibels (dB), at various positions is proportional to the logarithm of the distance from the source. In the simplest case, RSS can be expressed as:

$$RSS = P_t - K - 10\alpha \log_{10} d \tag{2.1}$$

Here, α is the path loss exponent. P_t is the transmit power. K is a constant that depends on the environment and the signal frequency.

RSS can be used to estimate the distance between a mobile device and an AP. However, the fluctuation of RSS is large in real environments, so distance estimation from a single RSS value is usually not accurate or reliable.

If a mobile device can receive signals from several transmitters (for example, multiple Wi-Fi APs), or if several fixed base stations can all detect the same device, we can build an RSS vector. This vector contains the signal strength from each AP or base station. Different locations produce different RSS vectors, which act like a “fingerprint” for that location. This is the basic idea of the Wi-Fi fingerprint positioning method used in this study.

Most modern Wi-Fi network adapters support measurement of RSS values from surrounding access points, even if performed through sequential scanning. In typical indoor settings,

mobile devices can generally detect signals from several APs simultaneously, making the use of multi-AP RSS data as a positioning fingerprint both technically feasible and practically valuable. The remainder of this discussion will focus on this fingerprinting approach.

Chapter 3

Theory

3.1 Obtain RSS data through simulation

In free space, there are no obstacles. Electromagnetic waves spread out evenly in all directions from the source. They form spherical waves.

Under these ideal conditions, propagation is the same in every direction. The signal power follows the inverse square law. This means the received power P is inversely proportional to the square of the distance d:

$$P \propto \frac{1}{d^2}.$$

This simple model is the basis of the Free-Space Path Loss (FSPL) model. It helps us understand how signal strength changes with distance. Later, we can compare this ideal case with real environments, such as underground mines, where obstacles, reflections and absorption make the signal behave very differently.

Received Signal Strength (RSS) is essentially the received signal power. In practice, we usually measure attenuation in decibels (dB). Therefore, it is more convenient to describe the relationship between RSS and distance in logarithmic form. In many environments, RSS decreases roughly in proportion to the logarithm of distance. The distance formula can be shown below.

$$RSS(d) = RSS(d_0) - 10n\log_{10}\left(\frac{d}{d_0}\right),$$

where n is called the path loss exponent. It describes how fast the signal decays in a given environment. In free space, $n = 2$. This is the most basic form of the logarithmic distance path loss model.

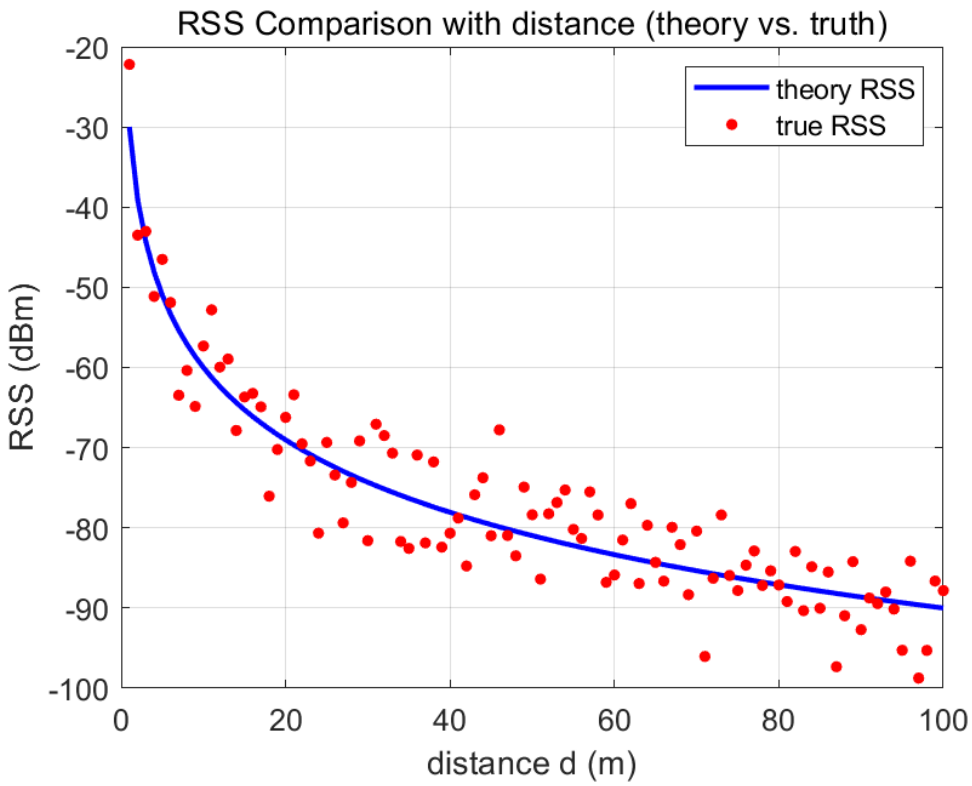

Figure 3.1: Simulation result of RSS

The result of simulation is shown in Figure 3.1

A simulated RSS environment is constructed by defining parameters such as room dimensions and access point (AP) locations. Ray tracing is employed to calculate the RSS value at each predefined location point. To ensure high spatial resolution, the sampling interval between points is set to a fine granularity (e.g., 0.01 meters). The results from a single ray tracing computation are stored as a reusable dataset, enabling all subsequent RSS-related analyses to efficiently retrieve signal strength values from this precomputed "RSS Simulation Environment Dataset" without repeated simulation.

3.2 Channel and PHY Error Model

We model the wireless channel using a log-distance large-scale path loss with constant-speed propagation delay and thermal noise. Bit/packet error behavior follows the analytical approximations of the ns-3 *YansErrorRateModel* under the configured MCS levels. Unless otherwise noted, default parameters are given in Table 3.1.

3.2.1 Path Loss and Received Power

For any transmitter–receiver separation $d \geq d_0$ (we use $d_0 = 1$ m), the large-scale path loss is

$$PL(d)\,[\text{dB}] = PL(d_0) + 10\,n \log_{10}\left(\frac{d}{d_0}\right). \tag{3.1}$$

The received power (used as an RSSI proxy) is

$$P_r(d)\,[\text{dBm}] = P_t[\text{dBm}] + G_t[\text{dB}] + G_r[\text{dB}] - PL(d)[\text{dB}]. \tag{3.2}$$

We adopt $PL(1\,\text{m}) = 8\,\text{dB}$, a path-loss exponent $n = 3.76$, $P_t = 0\,\text{dBm}$, and $G_t = G_r = 0\,\text{dB}$. If log-normal shadowing is required, one may add $X_\sigma \sim \mathcal{N}(0, \sigma^2)$ to (3.1); it is disabled in our baseline.

3.2.2 Noise Power and SNR

With bandwidth B (Hz) and receiver noise figure NF (dB), the thermal noise power is

$$N[\text{dBm}] = -174\ \text{dBm/Hz} + 10\log_{10} B + NF. \tag{3.3}$$

Hence the signal-to-noise ratio is

$$\text{SNR}_{\text{dB}} = P_r - N, \qquad \gamma = 10^{\text{SNR}_{\text{dB}}/10}. \tag{3.4}$$

For $B = 1$ MHz and $NF = 6.8\,\text{dB}$, $N \approx -107.2\,\text{dBm}$.

3.2.3 Propagation Delay

We use constant-speed propagation delay:

$$\tau(d) = \frac{d}{c}, \qquad c \approx 3 \times 10^8\ \text{m/s}, \tag{3.5}$$

without explicit multipath delay spread modeling.

3.2.4 MCS-Dependent Error Model (YansErrorRate)

For OFDM subcarriers using linear modulations, Yans first estimates the bit error rate (BER) from SNR and then maps it to packet error rate (PER) under an independent-bit ap-

Table 3.1: Baseline channel and receiver parameters

Parameter	Symbol	Value / Setting
Reference path loss (1 m)	$PL(d_0)$	8 dB
Path-loss exponent	n	3.76
Transmit power	P_t	0 dBm
Antenna gains	G_t, G_r	0 dB, 0 dB
Bandwidth	B	1 MHz
Noise figure	NF	6.8 dB
Noise power	N	≈ -107.2 dBm (from (3.3))
Propagation speed	c	3×10^8 m/s
Shadowing / Fast fading	—	Disabled (baseline)
MCS / GI / LDPC	—	Specified per experiment

proximation. For common modulations:

$$\mathrm{BER}_{\mathrm{BPSK}} = Q\left(\sqrt{2\gamma}\right) = \tfrac{1}{2}\,\mathrm{erfc}\left(\sqrt{\gamma}\right), \tag{3.6}$$

$$\mathrm{BER}_{\mathrm{QAM}} \approx \frac{4}{\log_2 M}\left(1 - \frac{1}{\sqrt{M}}\right) Q\left(\sqrt{\frac{3\log_2 M}{M-1}\gamma}\right), \tag{3.7}$$

where $Q(\cdot)$ is the Q-function and M is the QAM order. For a frame of N_{bits} bits,

$$\mathrm{PER} \approx 1 - (1 - \mathrm{BER})^{N_{\mathrm{bits}}}. \tag{3.8}$$

Coding gain and implementation specifics per MCS are embedded as effective thresholds/adjustments in the ns-3 Yans implementation.

3.2.5 Topology for Reproducibility

To match our simulations, we linearly place three APs at $x \in \{0, 500, 1000\}$ m (with $y = 0$) and deploy 32 STAs evenly over $x \in [0, 1000]$ m (with $y = 0$). For each STA, P_r, SNR, BER/PER, and delay follow ; MAC-layer RAW/TIM scheduling and contention/retx shape the net latency and throughput.

3.3 DQN learning

DQN, or Deep Q-Network, remains fundamentally a Q-learning algorithm. Its core principle lies in making the estimated Q-value as close as possible to the actual Q-value, or rather, ensuring the predicted Q-value under the current state aligns closely with the Q-value derived from past experience. In subsequent discussions, the actual Q-value is also referred to as the TD Target. The model is shown in figure 3.2.

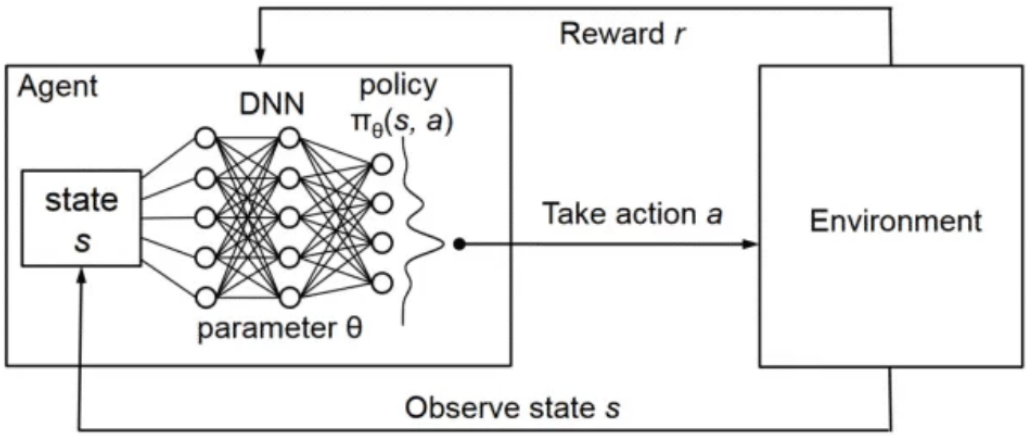

Figure 3.2: Model of DQN

Q-learning is a value–function–based reinforcement learning algorithm. It learns the state–action value function $Q(s,a)$ (Q-value) to guide the behaviour of the agent. The Q-value represents the expected long-term cumulative reward that the agent can obtain by taking action a in state s and thereafter following a certain policy. The core idea of Q-learning is to use a dynamic-programming–style update to iteratively improve the Q-values until they converge to the optimal action–value function Q^*.

The Q-learning update rule is

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right], \tag{3.9}$$

where α is the learning rate, γ is the discount factor, r is the immediate reward, s' is the next state, and $\max_{a'} Q(s',a')$ is the maximum estimated value of the next state over all possible actions a'.

In this update rule, α is the learning rate, which controls the step size of each update; γ is the discount factor, measuring the present value of future rewards; r is the immediate reward obtained by taking the current action; s' denotes the next state reached after executing action

a; and $\max_{a'} Q(s', a')$ represents the maximum Q-value over all possible actions a' in the next state s'.

Experience replay is a key technique in DQN. It stores the agent–environment interaction experiences, i.e., tuples of state, action, reward, and next state $\langle s, a, r, s' \rangle$. During training, a small batch of experiences is randomly sampled from this replay buffer to update the network parameters, which effectively mitigates the data correlation problem in the deep learning training process.

Chapter 4

Experiment and simulation

4.1 Test the accuracy of Wi-Fi Halow location

In this project, the Wi-Fi signal environment is first initialized by configuring the positions of Access Points (APs). For positioning purposes, an appropriate path loss model or RSSI fingerprint matching method is selected. Regarding signal propagation modeling, the Log-Distance Path Loss Model is adopted to characterize the relationship between signal attenuation and distance in the underground environment, whose formula is:

$$P(d) = P_0 - 10n\log_{10}(d) + X_g$$

In this model, $P(d)$ denotes the received signal strength (in dBm) at distance d is the reference signal strength measured at a reference distance (typically 1 meter), n is the path loss exponent (usually ranging between 2 and 4), and X_g represents a Gaussian random variable accounting for shadowing effects.

For positioning, two primary approaches are considered: RSSI fingerprinting and trilateration. In the RSSI fingerprinting method, a database is constructed by collecting WiFi RSSI measurements at multiple known locations, which is subsequently used for real-time location estimation via pattern-matching algorithms such as K-Nearest Neighbors (KNN), Support Vector Machine (SVM), or neural networks. In the trilateration approach, the user's location is estimated based on distance estimates derived from RSSI values measured from at least three access points.

To evaluate positioning accuracy, the Root Mean Square Error (RMSE) is adopted as the

performance metric, calculated as

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}((x_i - x_{\text{true}})^2 + (y_i - y_{\text{true}})^2)}$$

Here, x_i and y_i are the expected result, x_{true} and y_{true} are the true position, N is the number of the example.

There are two kinds of condition in simulation. The first one is that all APs are on one chain. It is similar to the real condition in the tunnel, which is shown in the Figure 4.1.

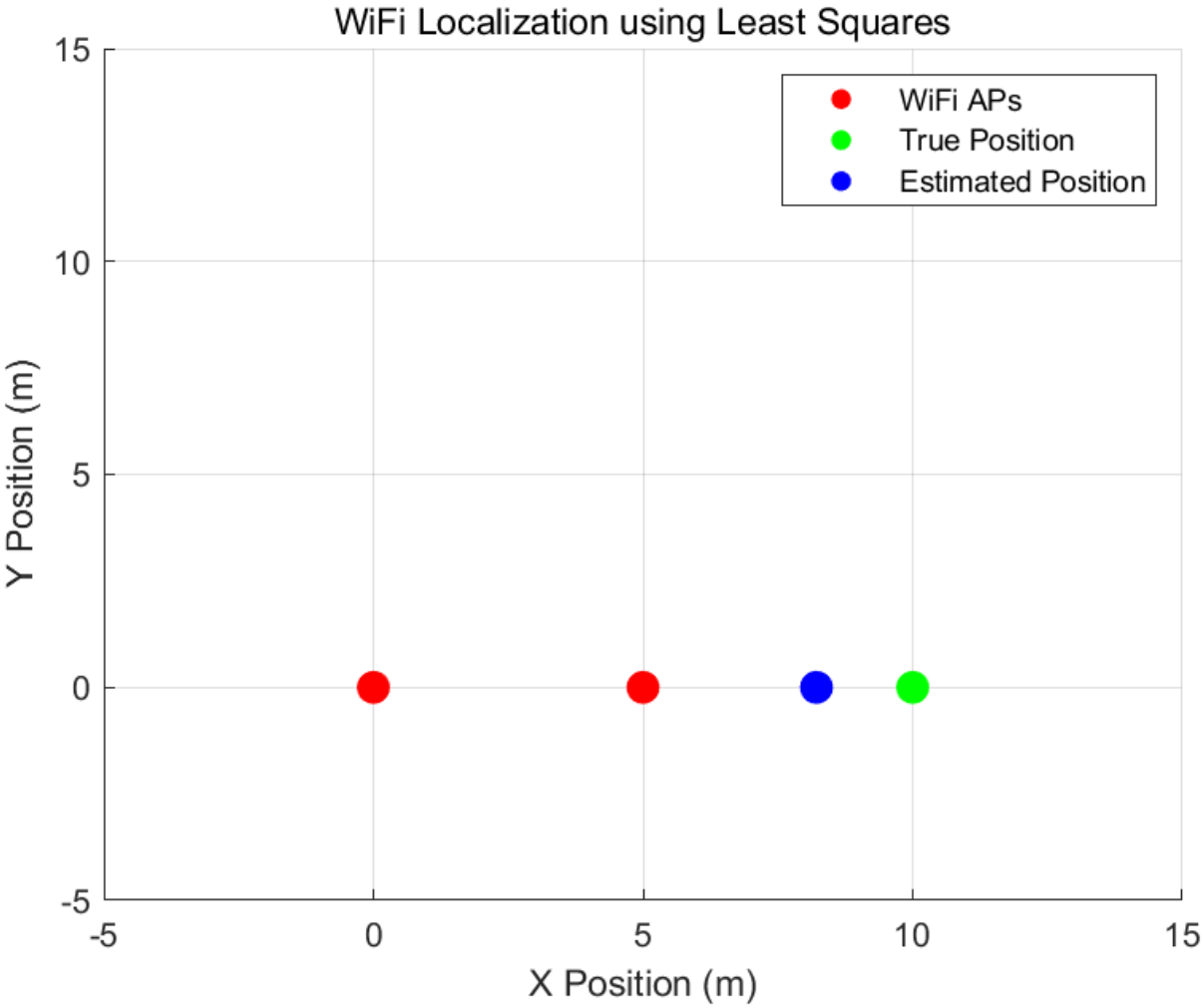

Figure 4.1: One line model

The least squares method is a widely used mathematical optimization technique for determining the best-fit solution from data containing errors. Its fundamental principle is to minimize the sum of squared residuals between observed values and model-predicted values, thereby obtaining a solution that most closely aligns with the overall trend of the dataset.

In positioning applications, precise distance measurements are often unavailable, preventing direct calculation of exact user locations. The least squares method addresses this challenge

by constructing a system of equations incorporating multiple sets of approximate distance measurements. It systematically reconciles the constraints imposed by these error-containing distances to derive an estimated position that demonstrates optimal consistency with all measured distance values.

In our implementation, this method is specifically applied to compute user locations by processing the coordinates and estimated distances from multiple WiFi access points. The mathematical realization is achieved through matrix operations as follows:

$$\hat{x} = (A^T A)^{-1} A^T b$$

We are given a set of data points (x_i, y_i). We want to approximate them with a straight line

$$y = ax + b.$$

In practice, the points do not lie exactly on a single line. Therefore, we choose a and b to minimise the total squared error:

$$\min_{a,b} \sum_i \left(y_i - (ax_i + b)\right)^2.$$

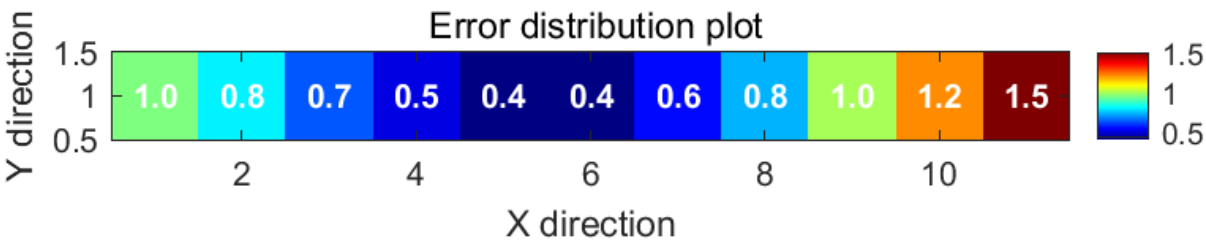

Figure 4.2: Error distribution plot

The result is shown in Figure 4.2. From the graph, it is shown that when it connects more APs, it is more correct.

The second condition is that there are three Aps. The model is shown in Figure 4.3 Trilateration is a localization technique that estimates a target's position using distance measurements from multiple reference points with known coordinates—such as WiFi access points, base stations, or satellites. By leveraging geometric relationships, the method determines the location where these distances concurrently converge.

In a two-dimensional plane, at least three reference points are required. Each point serves as the center of a circle whose radius corresponds to the measured distance to the target. Ideally, these circles intersect at a single point representing the target's location. In real-world

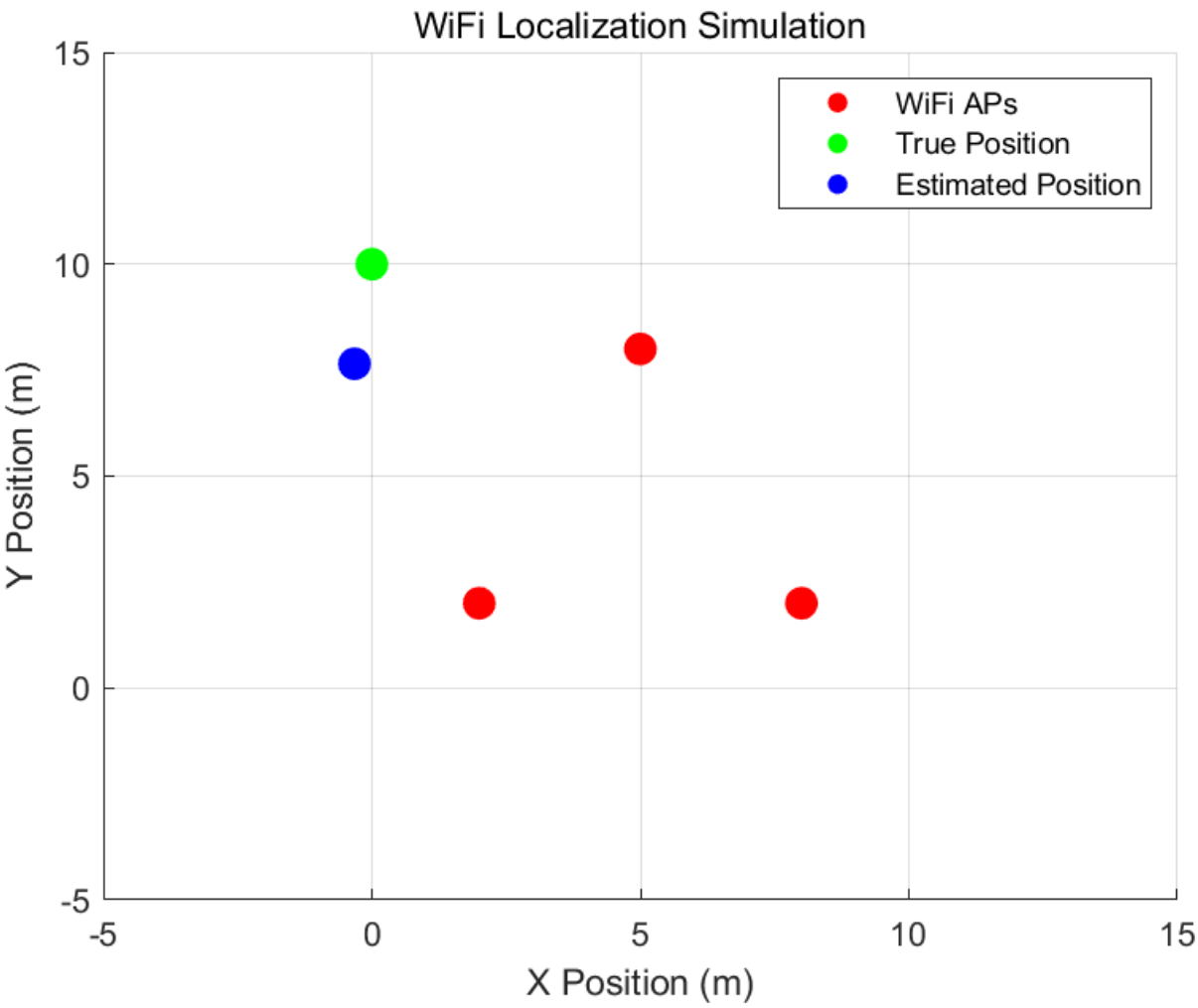

Figure 4.3: Three Aps in triangle shape

scenarios, however, measurement inaccuracies often result in an overlapping region rather than a perfect intersection. In such cases, estimation algorithms—such as least squares optimization—are applied to determine the most probable position within the overlapping zone.

This approach forms the underlying principle for many widely used positioning systems, including GPS and WiFi-based localization, where multilateration (its 3D equivalent) is employed to estimate user coordinates in physical space. In a two-dimensional space, suppose there are three known reference points,(x_1, y_1), (x_2, y_2) and (x_3, y_3). The unknown position of the target point is (x, y), and the distances between these three reference points can be measured, respectively as follows:

$$(x - x_1)^2 + (y - y_1)^2 = d_1^2$$

$$(x - x_2)^2 + (y - y_2)^2 = d_2^2$$

$$(x - x_3)^2 + (y - y_3)^2 = d_3^2$$

The result is shown in Figure 4.4

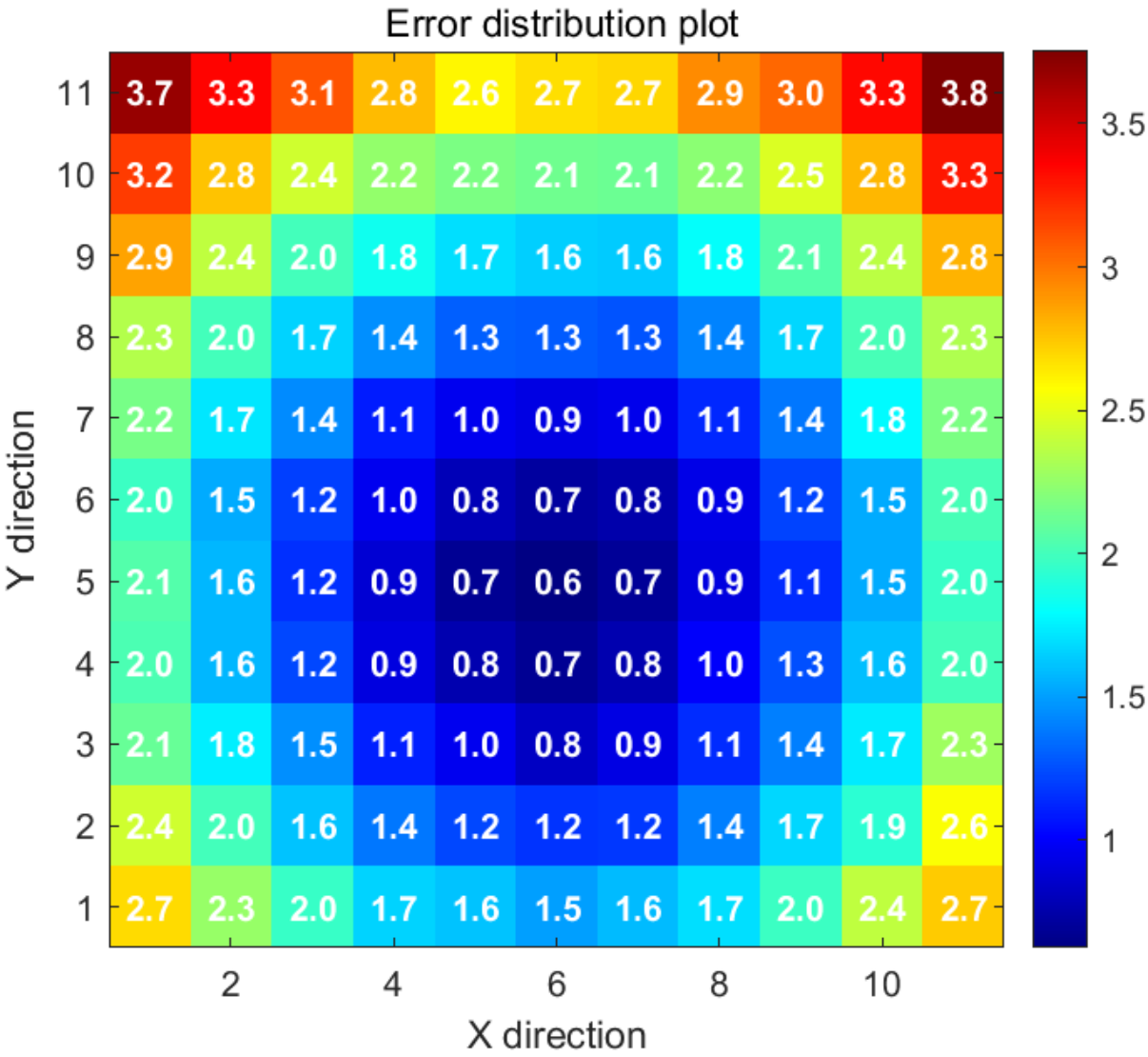

Figure 4.4: Error distribution plot

The central region (X = 6–7, Y = 6–7) exhibits the lowest positioning error, ranging between 0.6 and 0.9 meters. This suggests optimal Wi-Fi signal coverage and highest localization accuracy near the center of the environment.

In contrast, edge areas such as X = 1 or 11 and Y = 1 or 11 show significantly higher errors, between 2.6 and 3.8 meters. This can be attributed to increased signal attenuation and stronger multipath interference near boundaries, which degrade ranging reliability and positioning precision.

The error distribution appears roughly symmetric and radially increasing, consistent with typical Wi-Fi signal propagation behavior in open or regularly shaped environments such as tunnels and large indoor spaces. Errors are minimized near the center and gradually escalate

with distance from it, forming a gradient-like pattern that aligns with expected path loss and geometry-based dilution of precision.

4.2 Experiment

4.2.1 RSSI measurement preparation in hardware

The hardware is shown in the figure 4.5. The HW-9066 is a wireless module. It supports

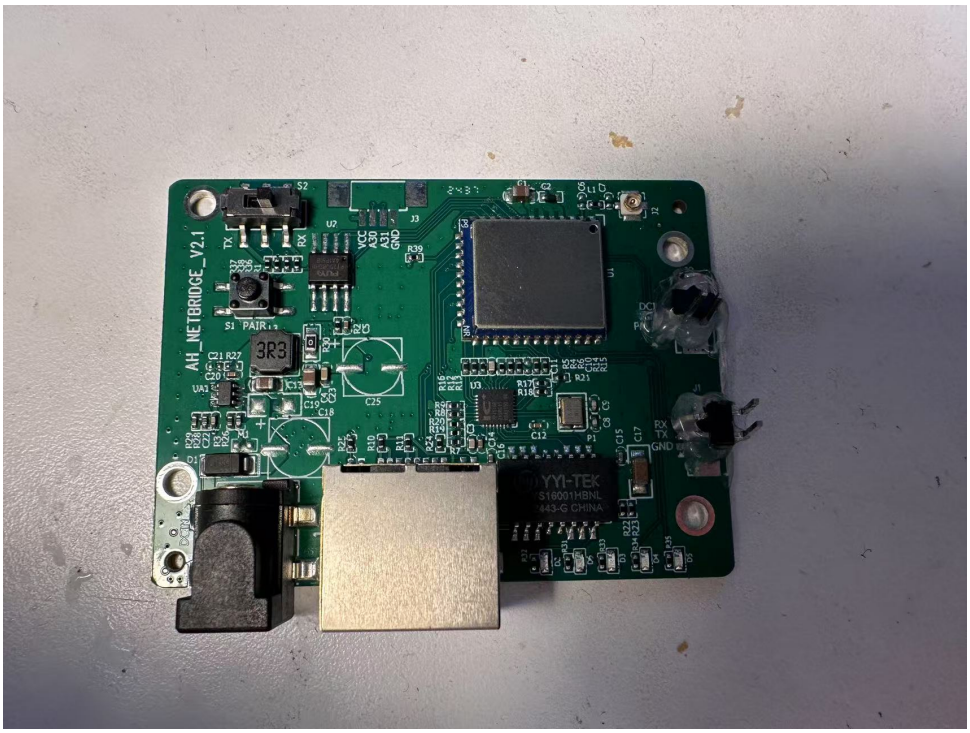

Figure 4.5: Picture of HW-9066

Wi-Fi HaLow (IEEE 802.11ah). It is mainly used for long-distance image and data transmission in unmanned aerial vehicles (UAVs).

The HW-9066 is a Wi-Fi HaLow (IEEE 802.11ah) wireless transmission module designed for long-range image and data transmission in unmanned aerial vehicle (UAV) applications.

We implemented a Python script for real-time serial data acquisition, which continuously read and parsed RSSI values from the serial interface at one-second intervals. Measurements were conducted at multiple representative locations including classrooms, corridors, stairwells, and outdoor open areas. At each point, ten RSSI samples were collected and averaged to mitigate temporal fluctuations. The resulting dataset establishes a correlation between signal

strength and physical location, providing a foundation for subsequent path loss modeling and localization algorithm validation.

4.2.2 Test environment

In the experiments, Wi-Fi HaLow APs and STAs are placed inside this 120m corridor system. There are three levels. Each level is separated by concrete slabs, which is about 0.2m thick. We create LOS and NLOS propagation paths, which are used to study signal strength, MCS, and packet performance in a tunnel-like environment. The structure is shown in figure 4.6.

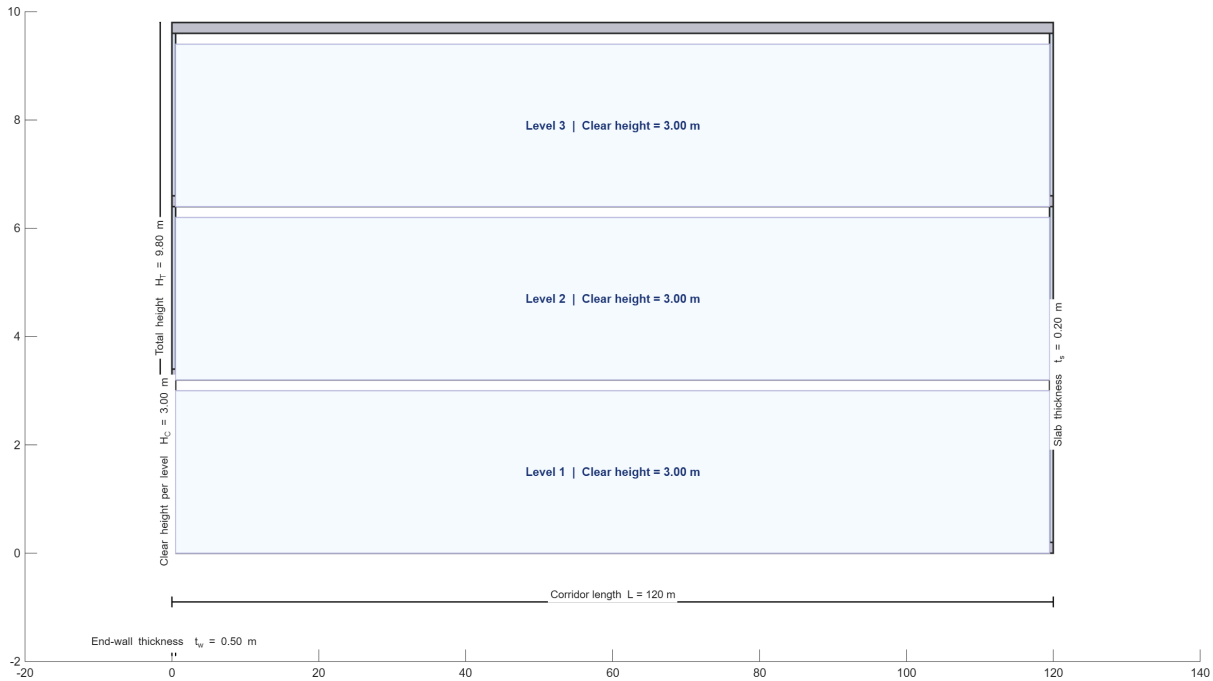

Figure 4.6: Test environment structure

4.2.3 Measurement results and discussion

In Los conditions, AP and STA were placed on the same corridor level with an unobstructed propagation path. In NLoS conditions, AP and STA were located on the same level but separated by internal walls so that direct visual contact is blocked. In Wall(cross-floor): AP and STA were located on different levels, with at least one reinforced-concrete floor slab between them to evaluate vertical penetration loss.

At each test position and for each scenario, a measurement run was performed as follows:

The STA was placed at the target distance and orientation and kept static during the test.

A sequence of ICMP/UDP packets with fixed payload size was transmitted between the STA and AP for a predefined duration.

During the run, the STA logged per-packet RSSI and SNR values reported by the radio, as well as timestamps of transmitted and received packets.

The results of the test is shown in figure 4.7 4.8 4.9 4.10.

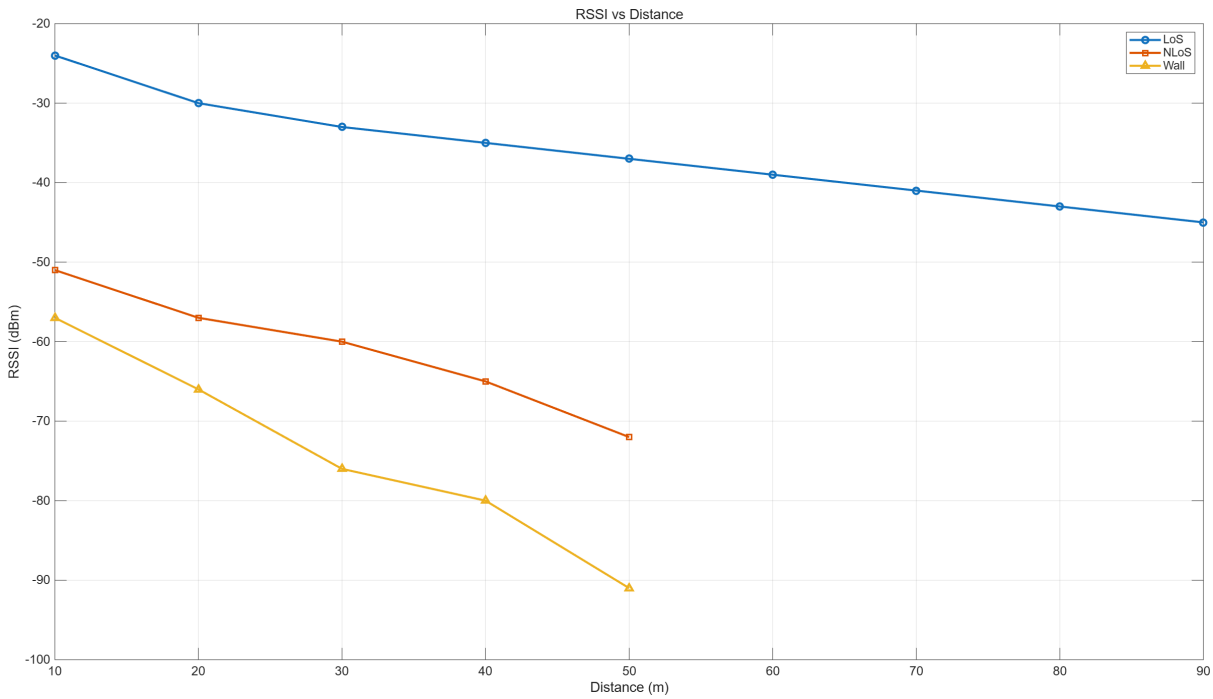

Figure 4.7: RSSI vs. Distance

RSSI versus distance

Figure 4.7 plots the average RSSI versus distance. In the LoS case, the RSSI decreases smoothly from approximately -23 dBm at 10 m to about -46 dBm at 90 m, which is consistent with a logarithmic distance-dependent path-loss model in a waveguide-like corridor. The NLoS curve is shifted downwards by roughly 20–25 dB compared with LoS at the same distances, starting around -50 dBm at 10 m and dropping to about -67 dBm at 50 m. The *Wall* scenario exhibits the strongest attenuation: the RSSI is already about -57 dBm at 10 m and falls below -90 dBm at 50 m, approaching the receiver sensitivity limit. These results confirm that internal corners and especially floor slabs introduce substantial excess loss on top of the distance-dependent path loss.

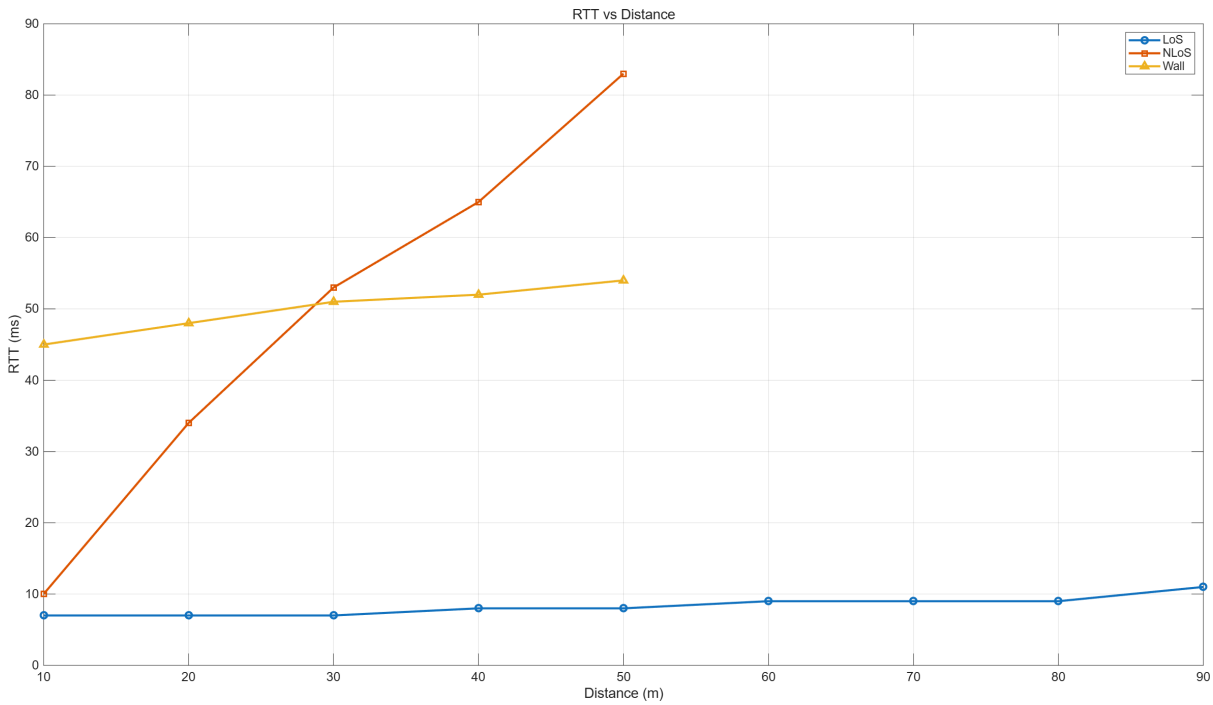

Figure 4.8: RTT vs. Distance

RTT versus distance

The RTT results in Fig. 4.8 reveal how link quality impacts latency. For LoS, the RTT remains almost constant between 7 and 10 ms across the entire range up to 90 m, indicating very few MAC-layer retransmissions and stable modulation and coding scheme (MCS) selection. In contrast, the NLoS RTT increases rapidly with distance: from roughly 10 ms at 10 m to over 80 ms at 50 m. Similarly, the *Wall* scenario exhibits elevated RTT even at short distances, on the order of 45–55 ms. Since the propagation delay itself is negligible at these scales, the RTT growth is primarily attributed to an increased number of retransmissions and backoff periods, as the link layer attempts to compensate for the higher error probability on degraded channels.

SNR versus distance

Figure 4.9 shows the evolution of SNR with distance. Under LoS conditions, the SNR starts around 46 dB at 10 m and remains above 20 dB even at 90 m, providing a generous link margin for the considered MCS. Both NLoS and *Wall* curves start at approximately 34 dB at 10 m, but decrease more rapidly: at 50 m the SNR drops to around 16 dB for NLoS and 14 dB for the *Wall* case. These values are close to typical demodulation thresholds for low-order MCSs, and the corresponding PER results in Fig. 4.10 confirm that such SNR levels are insufficient to

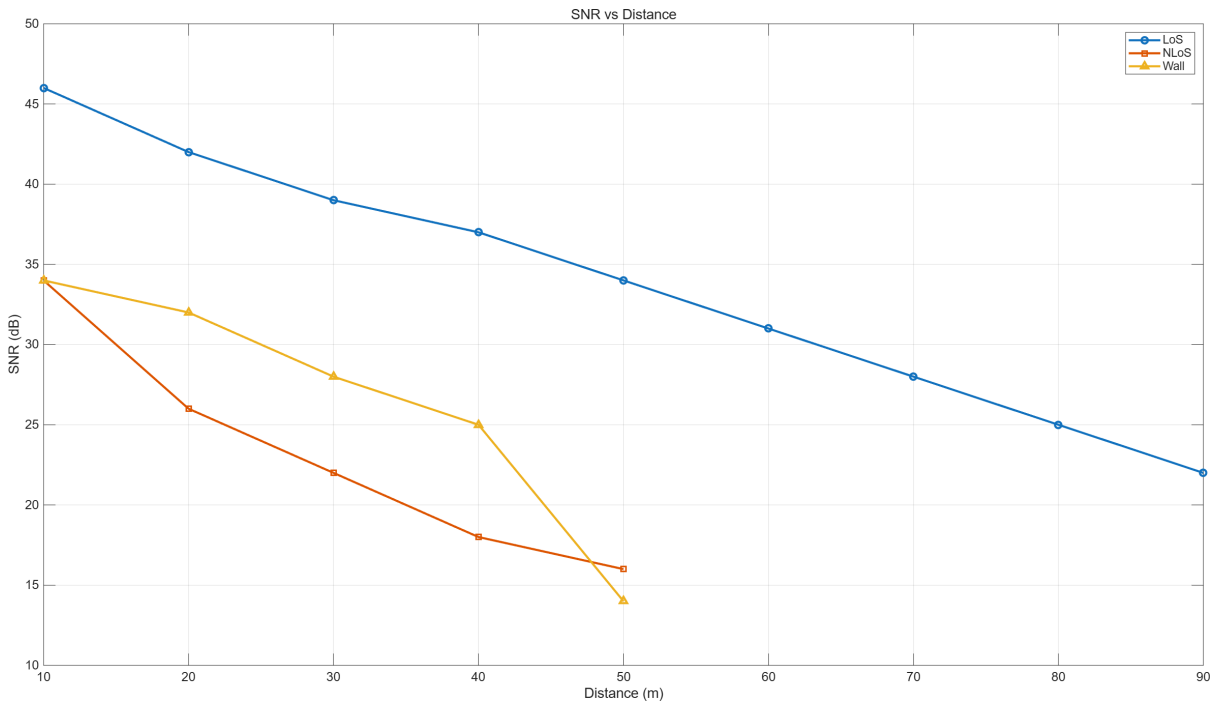

Figure 4.9: SNR vs. Distance

maintain reliable communication in the presence of fading.

PER versus distance

The PER behaviour in Fig. 4.10 clearly differentiates the three scenarios. For LoS, the PER is essentially 0 % from 10 m to 70 m and only increases moderately to about 5 % at 80 m and 12 % at 90 m. This indicates a wide range of distances over which the link is practically error-free. In the NLoS case, the PER rises sharply with distance: from near 0 % at 10 m to approximately 25 % at 20 m, 62 % at 30 m, and more than 90 % at 50 m. The *Wall* scenario is the most challenging, with PER already exceeding 90 % at 10 m and approaching 100 % for distances beyond 20 m. These results suggest that, with the current configuration, cross-floor links are effectively unusable for reliable data transfer.

MCS selection as a function of RSSI

Figures **??** and **??** show the empirical relationship between the modulation and coding scheme (MCS) selected by the Wi-Fi HaLow devices and the measured RSSI. For clarity, the horizontal axis is expressed as $|\text{RSSI}|$ in dBm, so that smaller values correspond to stronger received power (e.g., 70 on the axis denotes RSSI ≈ -70 dBm).

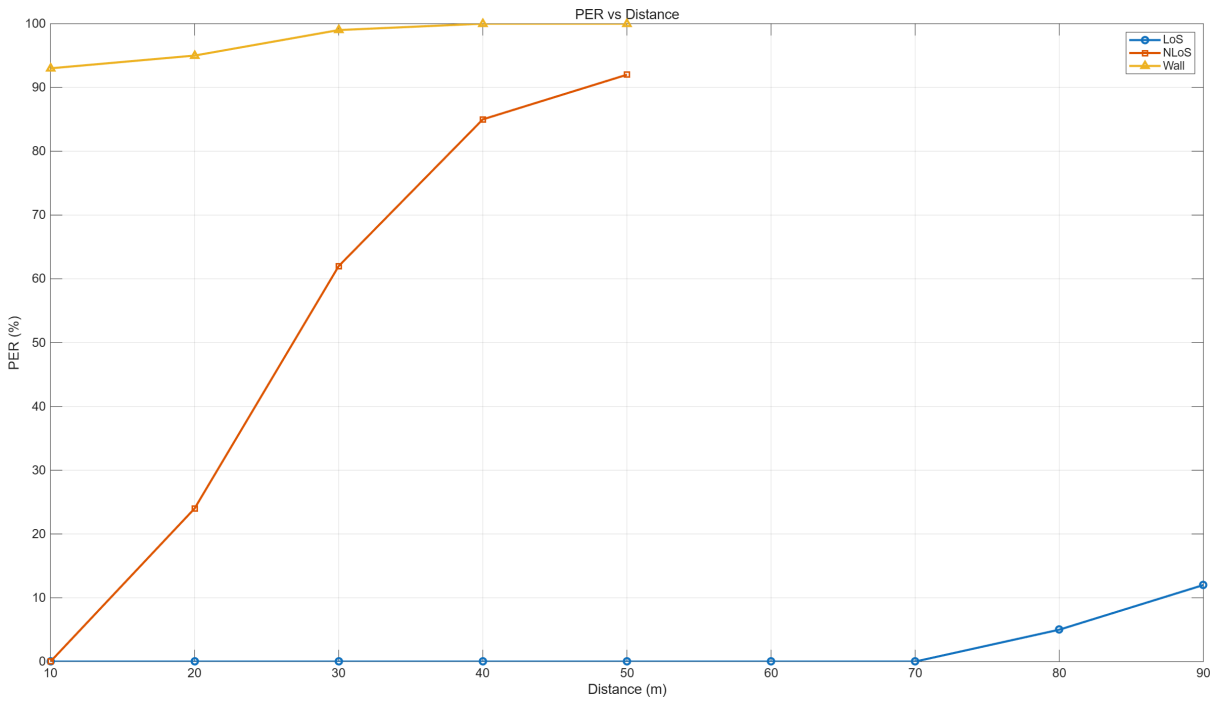

Figure 4.10: PER vs. Distance

8 MHz bandwidth. In Fig. 4.11, the MCS index exhibits a staircase behaviour with respect to $|\text{RSSI}|$. For strong signals ($|\text{RSSI}| \lesssim 78$ dBm), the PHY layer consistently selects the highest available mode, MCS $= 6$. As the receive power decreases, the selected MCS is downgraded in discrete steps to preserve reliability. Approximately, the transition regions can be summarised as

$$
\begin{aligned}
\text{MCS} &= 6, \quad |\text{RSSI}| \lesssim 78 \text{ dBm}, \\
\text{MCS} &= 5, \quad 78 \text{ dBm} \lesssim |\text{RSSI}| \lesssim 81 \text{ dBm}, \\
\text{MCS} &= 4, \quad 81 \text{ dBm} \lesssim |\text{RSSI}| \lesssim 84 \text{ dBm}, \\
\text{MCS} &= 3, \quad 84 \text{ dBm} \lesssim |\text{RSSI}| \lesssim 87 \text{ dBm}, \\
\text{MCS} &= 2, \quad 87 \text{ dBm} \lesssim |\text{RSSI}| \lesssim 90 \text{ dBm}, \\
\text{MCS} &= 1, \quad 90 \text{ dBm} \lesssim |\text{RSSI}| \lesssim 92 \text{ dBm}, \\
\text{MCS} &= 0, \quad |\text{RSSI}| \gtrsim 92 \text{ dBm}.
\end{aligned}
$$

This “cliff-like” mapping indicates that small changes in RSSI around the thresholds can cause abrupt changes in the data rate.

2 MHz bandwidth. Figure 4.12 presents the same measurement for a 2 MHz channel. The staircase shape is preserved, but all transition points are shifted by approximately 1–2 dB to-

wards stronger signal levels. In other words, a slightly higher receive power is required to sustain the same MCS when the bandwidth is doubled, which is consistent with the higher noise floor of a wider channel. The approximate regions for the 2 MHz case are

$$\begin{aligned}
\text{MCS} = 6, &\quad |\text{RSSI}| \lesssim 76\text{–}77 \text{ dBm}, \\
\text{MCS} = 5, &\quad 77\text{–}80 \text{ dBm}, \\
\text{MCS} = 4, &\quad 80\text{–}83 \text{ dBm}, \\
\text{MCS} = 3, &\quad 83\text{–}86 \text{ dBm}, \\
\text{MCS} = 2, &\quad 86\text{–}89 \text{ dBm}, \\
\text{MCS} = 1, &\quad 89\text{–}92 \text{ dBm}, \\
\text{MCS} = 0, &\quad |\text{RSSI}| \gtrsim 92 \text{ dBm}.
\end{aligned}$$

Discussion and implications. These results confirm that the commercial HaLow devices implement a threshold-based link adaptation algorithm, where the selected MCS is a piecewise-constant function of RSSI. From a system-design perspective, two main conclusions can be drawn:

- For both 1 MHz and 2 MHz bandwidths, maintaining $|\text{RSSI}| \lesssim 80$ dBm (i.e., RSSI $\gtrsim -80$ dBm) ensures operation with high-order MCSs (MCS 5–6) and thus high throughput.
- When $|\text{RSSI}|$ approaches 90 dBm, the PHY rapidly drops to the lowest MCSs (0–1), which drastically reduces the data rate and increases latency, and for $|\text{RSSI}| \gtrsim 92$ dBm the link becomes marginal.

In subsequent simulation and optimisation work, this empirical MCS–RSSI relationship is used to calibrate the path-loss model and to design AP placements and routing strategies that keep most links above the measured MCS thresholds.

Implications for corridor and mine-tunnel deployment

Combining the four metrics leads to several practical insights for Wi-Fi HaLow deployment in corridor- and tunnel-like environments:

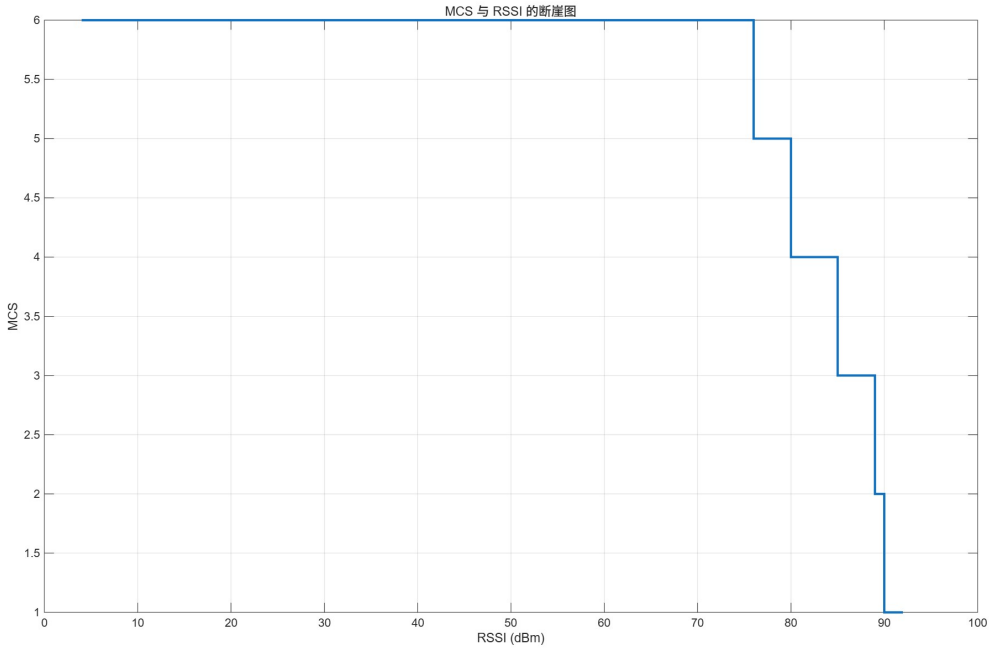

Figure 4.11: MCS vs. RSSI(8MHZ)

- Under LoS conditions, the AP–STA separation can be extended to approximately 60–70 m while maintaining high SNR, low RTT, and near-zero PER, which is suitable for typical mine-tunnel segments.

- Same-level NLoS links (e.g., around corners or behind internal walls) exhibit significantly reduced coverage. To keep PER below about 5 %, the effective distance should be limited to roughly 10–20 m, or additional relay nodes should be installed near bends and junctions.

- Cross-floor (*Wall*) communication through reinforced-concrete slabs shows extremely poor reliability, with PER above 90 % at all tested distances. Such links should not be used for critical traffic; instead, vertical shafts or stairwells should host dedicated relays, leaky feeders, or cabling solutions.

- The measurements indicate that reliable operation requires maintaining approximately RSSI $\gtrsim -45$ dBm and SNR $\gtrsim 25$ dB. These empirical thresholds can be used to calibrate the path-loss model in simulation and to guide AP placement and routing decisions in subsequent optimisation and reinforcement-learning studies.

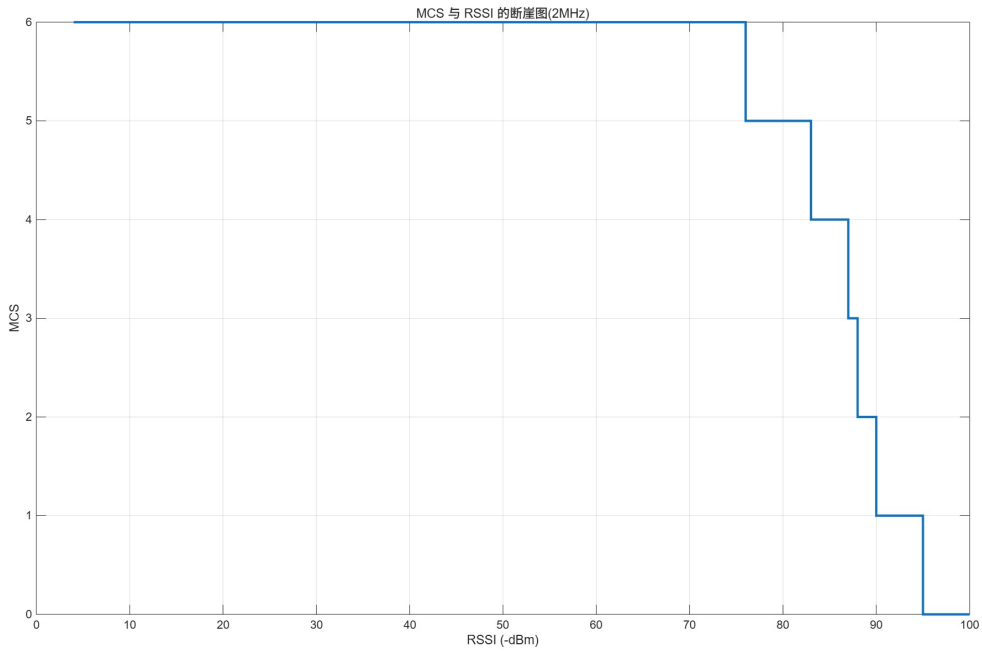

Figure 4.12: MCS vs. RSSI(2MHZ)

4.3 NS3 simulation

4.3.1 Build NS3 environment

In this project, I first installed and configured the NS-3 (Network Simulator 3) environment. This provided the basic platform for later wireless protocol simulations. NS-3 is a modular, discrete-event network simulator. It is written mainly in C++ and also provides Python bindings. Because it is distributed as source code, the setup is more complex than a normal software installation.

I began by installing the required development tools. These included `g++`, `python3-dev`, `cmake`, and `pkg-config`. I also installed graphical libraries such as Qt5, and Python-related tools like `pybindgen` and `setuptools`. All of these were installed using the Linux APT package manager. After that, I used the Waf build system to configure and compile NS-3. This required running `./ns3 configure` and `./ns3 build`, and fixing issues related to compiler versions, header paths, and optional modules.

Through this process, I learned the basic module structure and dependency management of NS-3. This knowledge is important for later experiments on Wi-Fi HaLow and other communication protocols.

To support graphical visualization, I also compiled the NetAnim animation tool. This step required a working Qt development environment and a check that NetAnim matched the NS-3 version. A further challenge was the integration of the IEEE 802.11ah (Wi-Fi HaLow) module. This module is maintained as a third-party patch. I cloned it from an external Git repository and then adapted it to my NS-3 version. This involved editing the build configuration and adjusting some source files to make sure the code could compile and run.

Overall, the full installation and configuration process showed the importance of a consistent environment and careful version control. NS-3 offers powerful simulation functions, but the initial setup has a steep learning curve and demands systematic troubleshooting on a Linux system.

4.3.2 NS3 environment test

During the installation, I ran the NS-3 configuration command in the terminal. The output showed: `'configure' finished successfully (1.358s).`

This message means that the basic dependencies and environment settings were found and checked. The system correctly detected the compiler and the Python interface. After this step, the process could safely move on to the build stage.

Therefore, although the configuration step looks quick and simple, it is an important milestone. It confirms that the environment is ready. It also ensures that later steps, such as compiling and running NS-3, will not be stopped by missing or mismatched dependencies.

The result is shown in the figure 4.13.

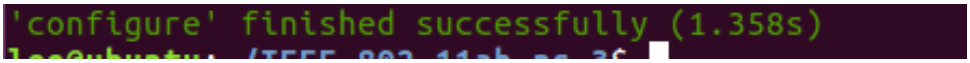

Figure 4.13: Result of configure

The terminal output shows that NS-3 checked many parts of the system. At the end, it printed the message `'configure' finished successfully`. This looks like a smooth setup. However, the same output also listed several modules that were turned off.

- Python bindings were disabled. The reason was a version mismatch of `pybindgen`. NS-3 needs version `0.17.0.887`, but the system had version `0.22.1`. The configure script did not try to downgrade or replace it.

- NS-3 Click integration and OpenFlow support were both disabled. This means advanced routing and SDN-related functions are not available by default. To use them, I would need to install extra libraries and enable them by hand.
- PyViz visualization was not enabled. As a result, real-time visual display of simulations is not supported in the current setup.
- Other components, such as Gcrypt, NSC (Network Simulation Cradle), and PlanetLab support, were also not configured. In most cases, this was due to missing packages or version conflicts.

This result shows that setting up NS-3 is not a simple one-click process. It needs careful checks of low-level tools and libraries. The modular design of NS-3 gives strong flexibility, but it also raises the technical requirements for the user. One must understand compilers, Python versions, library paths, and basic Linux system management.

By going through this configuration step, I gained a clearer view of how a large open-source simulation framework is built on top of the operating system. This experience also prepared the environment for later custom wireless network protocol simulations.

4.3.3 Build WiFi mesh network

Build the WiFi Halow node first. The code is shown in the figure 4.14.

```
NodeContainer wifiStaNode;
NodeContainer wifiApNode;
wifiStaNode.Create (1);
wifiApNode.Create (1);
```

Figure 4.14: Build WiFi node

The physical layer controls how wireless signals travel through space. It uses several models to imitate real-world conditions:

- **Path loss model:** Describes how signal strength decreases with distance. It is the basic tool for estimating the coverage range of a wireless link.

- **Shadowing:** Describes slow changes in received power caused by large obstacles, such as buildings, walls, or terrain. It represents large-scale variations.
- **Multipath fading:** Describes fast, small-scale changes in the signal. These changes come from constructive and destructive interference of multiple signal paths.

To simulate a wireless system accurately, these models must be chosen and set carefully. The model parameters should match the target environment, for example indoor, outdoor, urban, or rural. Proper configuration is essential for getting realistic and reliable performance results for each specific scenario.

```
MobilityHelper mob;
mob.SetMobilityModel ("ns3::ConstantPositionMobilityModel");
mob.Install (nodes);
nodes.Get (0)->GetObject<MobilityModel> ()->SetPosition (Vector (0, 0, 0));
nodes.Get (1)->GetObject<MobilityModel> ()->SetPosition (Vector (30, 0, 0));
```

Figure 4.15: Mine tunnel model

BreakDistance is the breakpoint distance. In this model it is set to 5 m. It marks the point where the path loss behaviour changes between two regions, for example near the tunnel entrance and deeper inside a mine tunnel.

- **Slope1:** This is the path loss exponent before the breakpoint. It usually takes a value between 1.5 and 2.0. In this near region, the tunnel acts like a waveguide. The walls help confine the signal, so the attenuation is relatively low.
- **Slope2:** This is the path loss exponent after the breakpoint. Typical values are between 3.0 and 4.0. In this far region, multipath scattering, reflections, and diffractions are stronger. The structure becomes more complex, and the signal decays faster.

Using two different slopes allows the model to better match real underground environments. It captures the change in propagation behaviour with distance from the source, which is important for wireless links in mine tunnels.

4.3.4 The result of simulation

Figure 4.17 shows the simulated IEEE 802.11ah signal-strength distribution in a corridor scenario with three access points (APs) and thirty-two stations (STAs). The APs are placed

```
channel.AddPropagationLoss ("ns3::TwoSlopePropagationLossModel",
                            "BreakDistance", DoubleValue (5.0),
                            "Slope1", DoubleValue (1.6),
                            "Slope2", DoubleValue (3.5));
```

Figure 4.16: Path loss

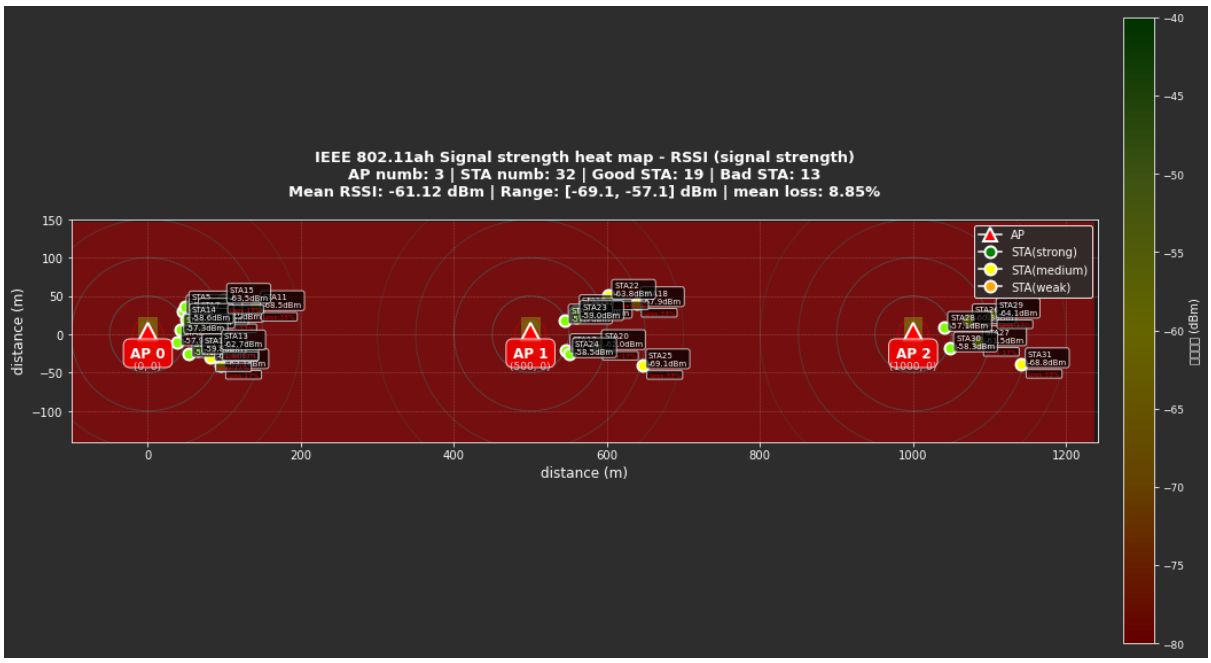

Figure 4.17: Result of the simulation

collinearly at $x = \{0, 500, 1000\}$ m on the corridor centreline ($y = 0$), while the STAs are distributed around these APs with small offsets in the y-direction. The background colour represents the received signal strength indicator (RSSI), with warmer colours denoting stronger signals and darker red indicating weaker coverage. Concentric rings around each AP indicate equal-distance contours.

The global statistics printed in the figure show an average RSSI of approximately -61.1 dBm, with values ranging from about -69.1 dBm to -57.1 dBm. Out of the 32 STAs, 19 are classified as "good" links and 13 as "bad" links, giving an estimated mean packet loss of about 8.85%. As expected, STAs located close to an AP are marked as strong/medium links and fall in the -57 to -62 dBm range, whereas STAs positioned near the midpoints between APs or near the corridor edges experience weaker RSSI (around -65 to -69 dBm). This spatial pattern confirms that the chosen AP spacing provides continuous coverage along the corridor, but also highlights transition regions where roaming or rate adaptation will be triggered.

From a design perspective, this result suggests that keeping the majority of user locations above roughly -62 to $-65\,\text{dBm}$ is feasible with the current three-AP layout, allowing most STAs to operate with medium–high MCS and stable throughput. However, the weaker zones close to the AP midpoints indicate that a roaming threshold around -65 to $-67\,\text{dBm}$ and a minimum acceptable floor above $-70\,\text{dBm}$ should be adopted. Below this level the simulation predicts a higher probability of low MCS, increased retransmits, and degraded QoS, which can be mitigated either by slightly adjusting AP positions or by adding an extra AP in longer or more obstructed corridors.

Chapter 5

Optimal mesh network path Planning

5.1 Design of the code

5.1.1 Reinforcement–Learning-Based Path Planning Implementation

In order to obtain a routing policy that is aware of the tunnel geometry and the Wi-Fi HaLow channel characteristics, we implement a Deep Q-Network (DQN) agent in Python using PyTorch. The complete code consists of four main components:

1. a mine tunnel communication environment `MineEnvironment`;
2. a dueling DQN model `DQNetwork`;
3. a replay buffer and DQN agent `DQNAgent`;
4. the training loop, performance logging, and path visualisation.

This section summarises the key design choices and how they encode the underground mine constraints.

5.1.2 Mine Tunnel Environment

The class `MineEnvironment` simulates an underground corridor with a main tunnel and multiple vertical branches. The main tunnel is modelled as a straight line along the x-axis with length

$$L_{\text{main}} = 1200 \text{ m},$$

and access points (APs) are placed every 50 m along this tunnel. Additional branch tunnels are defined via a configuration list.Its horizontal position x along the main tunnel. The existence of an upper branch (`has_up`) and/or a lower branch(`has_down`) are located along the main tunnel, whose length along $\pm y$.

For each branch, APs are again deployed every 50 m along the vertical branch direction. As a result, the AP layout $\{(x_i, y_i)\}$ contains:

- a set of *main-tunnel APs* with $y_i = 0$;
- a set of *branch APs* with fixed x_i and $y_i > 0$ (upper branches) or $y_i < 0$ (lower branches).

The start AP index can be chosen the leftmost main-tunnel AP, while the target AP is either specified or automatically selected as the rightmost main-tunnel AP.

There is a Tunnel-awareness and "no wall crossing" constraint design. To prevent physically impossible shortcuts through rock, we define a helper function that checks whether two positions (x_1, y_1) and (x_2, y_2) lie in the same tunnel: Both on the main tunnel: $|y_1| < 1$ and $|y_2| < 1$. Both on the same branch: $|x_1 - x_2| < 1$ and $\text{sign}(y_1) = \text{sign}(y_2)$. One on the main tunnel and one close to the branch junction (small $|y|$ around the branch x-coordinate).

If these conditions are not satisfied, the link between the two positions is treated as *through rock* with very strong additional attenuation.

5.1.3 Channel and Path-Loss Model

The Wi-Fi HaLow carrier frequency is set to $f = 900$ MHz and the transmit power to $P_{\text{tx}} = 30$ dBm. The environment uses a log-distance path-loss model with optional wall penetration loss. For a link of length d (in metres), the path loss is

$$\text{PL}(d) = \text{PL}(d_0) + 10\,n \log_{10}\left(\frac{d}{d_0}\right) + X_\sigma + L_{\text{wall}}, \tag{5.1}$$

with reference distance $d_0 = 1$ m and

$$\text{PL}(d_0) = 20\log_{10}(f) + 20\log_{10}(d_0) - 147.55. \tag{5.2}$$

The path-loss exponent n, shadowing term $X_\sigma \sim \mathcal{N}(0, \sigma^2)$, and wall loss L_{wall} depend on whether there is a rock barrier:

- *Same tunnel* (no wall): $n = 3.5$, $\sigma = 4$ dB, $L_{\text{wall}} = 0$.

- *Through rock*: $n = 5.5$, $\sigma = 8$ dB, $L_{\text{wall}} = 40$ dB.

The received signal strength indicator (RSSI) at the station (STA) from AP i is then

$$\text{RSSI}_i = P_{\text{tx}} - \text{PL}_i(d). \tag{5.3}$$

Links with d larger than a maximum range (here 1000 m) are clipped to a very weak level (e.g. -100 dBm) and treated as unreachable.

The signal-to-interference-plus-noise ratio (SINR) for AP i is computed as

$$\text{SINR}_i = 10\log_{10}\left(\frac{P_i}{\sum_{j\neq i} P_j + N_0}\right), \tag{5.4}$$

where $P_i = 10^{\text{RSSI}_i/10}$ is the signal power from AP i, P_j is the interfering power from other APs whose RSSI exceeds a threshold (e.g. -90 dBm), and $N_0 = 10^{\text{noise_floor}/10}$ corresponds to a noise floor of -95 dBm. If the effective signal power is extremely small, the SINR is clipped to a low value to avoid numerical issues.

5.1.4 State Representation

Each environment state is a 68-dimensional feature vector that encodes the STA position, AP geometry, and link quality. We use:

1. **Relative STA position to the target** (2 dims): $(\mathbf{x}_{\text{target}} - \mathbf{x}_{\text{STA}})$, normalised by L_{main}.
2. **Current AP absolute position** (2 dims): $(x_{\text{AP}}, y_{\text{AP}})$, normalised by $(L_{\text{main}}, L_{\text{branch}})$.
3. **Current AP position relative to the target AP** (2 dims): $(\mathbf{x}_{\text{target AP}} - \mathbf{x}_{\text{AP}})$, normalised by L_{main}.
4. **Current AP SINR** (1 dim): SINR, normalised by 50 and clipped to $[-1, 1]$.
5. **Top-K candidate APs** (with $K = 10$):
   - RSSI for each candidate (10 dims), mapped linearly from $[-100, 0]$ dBm to $[0, 1]$;
   - relative vector from the current STA position to each AP (20 dims);
   - distance from each AP to the target AP, normalised by L_{main} (10 dims);
   - absolute position of each AP, normalised as above (20 dims).

6. **Handover count** (1 dim): total number of AP switches in the current episode, normalised by a fixed maximum (e.g. 20) and clipped to $[0,1]$.

If fewer than K APs are reachable (RSSI above a threshold), the remaining slots are padded with zeros and maximum distances so that the state vector always has a fixed dimension.

5.1.5 Action Space and STA Motion Model

The action space is discrete with 10 actions: action 0 is keep the current serving AP; actions 1–9 switch to one of the top-9 reachable APs ranked by RSSI.

If an action refers to a candidate index that does not exist, it is treated as "no switch". After choosing the serving AP, the STA moves by at most 10 m per step: if its distance to the current AP exceeds a threshold, it moves directly towards that AP; otherwise it moves towards the global target position. But the direction is constrained to follow the tunnel structure: In the main tunnel ($|y| < 30$), motion is mostly along the x-axis, with small y adjustments. In a branch, if the final target lies in the main tunnel, the STA tends to return towards $y = 0$; otherwise it progresses along the branch. This procedure enforces a physically realistic, corridor-following trajectory instead of allowing the STA to "teleport" through walls.

5.1.6 Reward Design

The reward signal is shaped to favour short, reliable, and physically feasible paths while disallowing wall crossings:

- **Progress reward:**

$$r_{\text{prog}} = 2\big(d_{\text{old}} - d_{\text{new}}\big),$$

  where d_{old} and d_{new} are the STA distances to the target before and after the step.

- **Handover penalty:** each AP switch incurs a fixed penalty (e.g. -2) to discourage excessively frequent handovers.

- **SINR-based reward:** strong links are rewarded (e.g. $+5$ for SINR > 20 dB, $+2$ for SINR > 10 dB), while poor links are penalised (e.g. -5 for SINR < 0 dB, -10 for SINR < -10 dB).

- **Coverage penalty:** if the best available RSSI over all APs is weak (e.g. below -85 or -75 dBm), a strong penalty is applied, reflecting that the STA is in a shadowed or invalid region.
- **Wall-crossing penalty:** if the serving AP and the STA position are detected as not being in the same tunnel, an additional penalty is added to discourage physically impossible links.
- **AP proximity reward:** if the serving AP is close to the target AP (e.g. within 100 or 300 m), a small positive reward is given.
- **Same-tunnel bonus:** if the serving AP lies in the same tunnel as the target AP, a bonus is applied to guide the agent towards a realistic corridor route.
- **Terminal rewards:** reaching the target region (distance < 20 m) yields a large positive reward (e.g. $+100$), while exceeding the maximum step limit (e.g. 200 steps) results in an episode termination with a negative penalty (e.g. -50).

5.2 Dueling DQN Architecture

The DQN model `DQNetwork` is implemented in PyTorch and uses a dueling architecture with separate processing for geometric and signal features. Given a state vector $\mathbf{s} \in \mathbb{R}^{68}$, we split it into two parts. A *position feature* sub-vector of dimension 46 including STA–target position, current AP position, AP–target vector, top-K relative positions, top-K absolute positions; A *signal feature* sub-vector of dimension 22 including current SINR, top-K RSSI, top-K distances to target, normalised handover count.

The position features are passed through a multi-layer perceptron (MLP)

$$\mathbb{R}^{46} \to 128 \to 64,$$

and the signal features are passed through another MLP

$$\mathbb{R}^{22} \to 64 \to 32.$$

The resulting embeddings are concatenated and further processed by a fusion network

$$(64+32) \to 256 \to 256 \to 128,$$

Table 5.1: Key DQN hyperparameters used in training.

Hyperparameter	**Setting**
Optimiser	Adam, learning rate 1×10^{-4}
Discount factor	$\gamma = 0.99$
Mini-batch size	128
Replay buffer capacity	50,000 transitions
Exploration strategy	ε-greedy, initial $\varepsilon = 1.0$
	multiplicative decay $\varepsilon \leftarrow 0.995\varepsilon$ per episode,
	minimum $\varepsilon_{\text{min}} = 0.01$
Target network update frequency	Every 10 training steps
Gradient clipping	$\|\nabla\| \leq 10$

with ReLU activations and dropout layers to mitigate overfitting. On top of this fused representation, we implement the dueling streams:

$$A(\mathbf{s}, a) = f_{\text{adv}}(\mathbf{h}) \in \mathbb{R}^{|\mathcal{A}|}, \tag{5.5}$$

$$V(\mathbf{s}) = f_{\text{val}}(\mathbf{h}) \in \mathbb{R}, \tag{5.6}$$

where $\mathbf{h}$ is the fused feature vector. The final Q-values are computed as

$$Q(\mathbf{s}, a) = V(\mathbf{s}) + \left(A(\mathbf{s}, a) - \frac{1}{|\mathcal{A}|}\sum_{a'} A(\mathbf{s}, a')\right). \tag{5.7}$$

This dueling structure helps the network to separately learn state values and action-dependent advantages.

5.2.1 DQN Agent and Training Procedure

The `DQNAgent` maintains a policy network and a target network, both instances of `DQNetwork`. The target network parameters are periodically updated with the policy network weights. Experience tuples $(\mathbf{s}, a, r, \mathbf{s}', d)$ are stored in a replay buffer of fixed capacity and sampled in mini-batches for training.

Key hyperparameters is shown below.

For each training episode:

1. The environment is reset to the STArt position of the AP and the corresponding STA position.

2. At each step, an action is selected using the current ε-greedy policy.

3. The environment evolves according to the movement and channel model, and returns the next state, reward, and a termination flag.

4. The transition is pushed into the replay buffer; if enough samples are available, a mini-batch is drawn and the Double DQN update is applied:

$$a'_* = \arg\max_{a'} Q_\theta(\mathbf{s}', a'), \tag{5.8}$$

$$y = r + (1-d)\gamma Q_{\theta^-}(\mathbf{s}', a'_*), \tag{5.9}$$

$$L = \mathbb{E}\Big[\big(Q_\theta(\mathbf{s}, a) - y\big)^2\Big], \tag{5.10}$$

   where θ and θ^- denote the policy and target network parameters, respectively.

5. The policy network parameters are updated via backpropagation to minimise L, and the target network is updated at fixed intervals.

During training, per-episode statistics (total reward, path length, success indicator, and average loss) are recorded. After training, the best-performing model (highest episode reward) and the final model are saved for later evaluation.

5.2.2 Results of Training Curves and Path Visualization

Together, the four subplots show that the DQN agent gradually learns a stable and reliable routing policy.

In Episode rewards, at the beginning, the raw returns fluctuate strongly and are mostly negative, indicating that the agent frequently hits penalties. After roughly 150–200 episodes, the smoothed reward curve rises above zero and keeps increasing towards 1200–1500, showing that the agent has learned to reach the target AP more often while avoiding severely attenuated or illegal links. The variance of the raw rewards also becomes smaller, which indicates a more deterministic policy rather than random wandering.

In Episode lengths, early in training, almost all episodes last close to the maximum of 200 steps, meaning the agent often fails to find a good path and times out. As training progresses, the average length drops to about 170–180 steps, while still completing the task, which corresponds to shorter and more efficient paths. The spread in episode lengths increases after

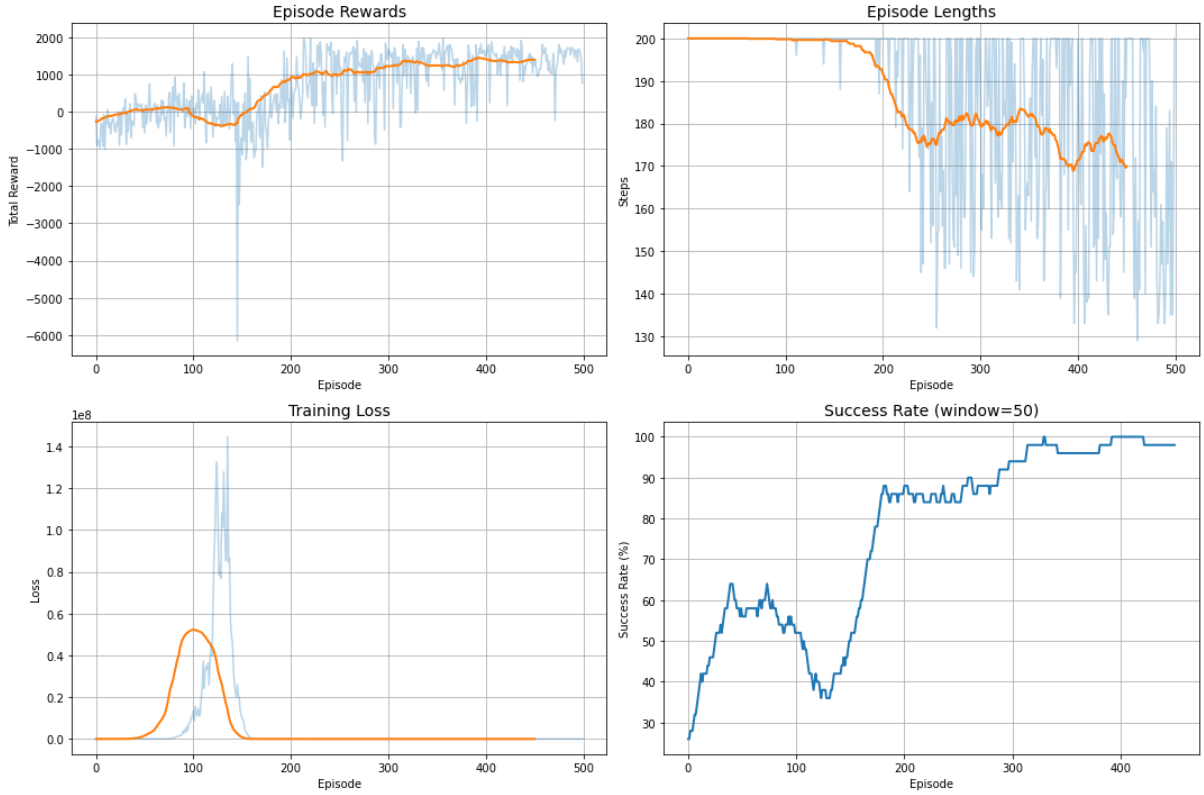

Figure 5.1: DQN training process

convergence because both successful short trajectories and occasional exploratory failures are present.

In Training loss, the loss exhibits a large spike between about 50 and 150 episodes, when the replay buffer is being filled and the Q-network is rapidly updated on diverse transitions. After this transient phase, the loss quickly decays towards (near) zero and stays low, suggesting that the Q-values have stabilised and the target and online networks are in agreement—i.e., the agent has essentially converged.

In Success rate, The initial success rate is around 20–30%, then rises to 60% by about 60 episodes. There is a dip around 80–120 episodes, which matches the high-loss phase and reflects aggressive exploration. After about 200–250 episodes, the success rate climbs to over 85% and eventually approaches 95–100%, showing that the final policy can almost always discover a valid path under the given tunnel/channel conditions.

Overall, these curves indicate stable learning and near-optimal performance: rewards go up, paths get shorter, the loss stabilises, and the success rate saturates close to 100%.

Figure 5.2 visualises the learned routing behaviour in the mine layout.

Blue squares denote all candidate APs deployed along the main drift and several upper/lower

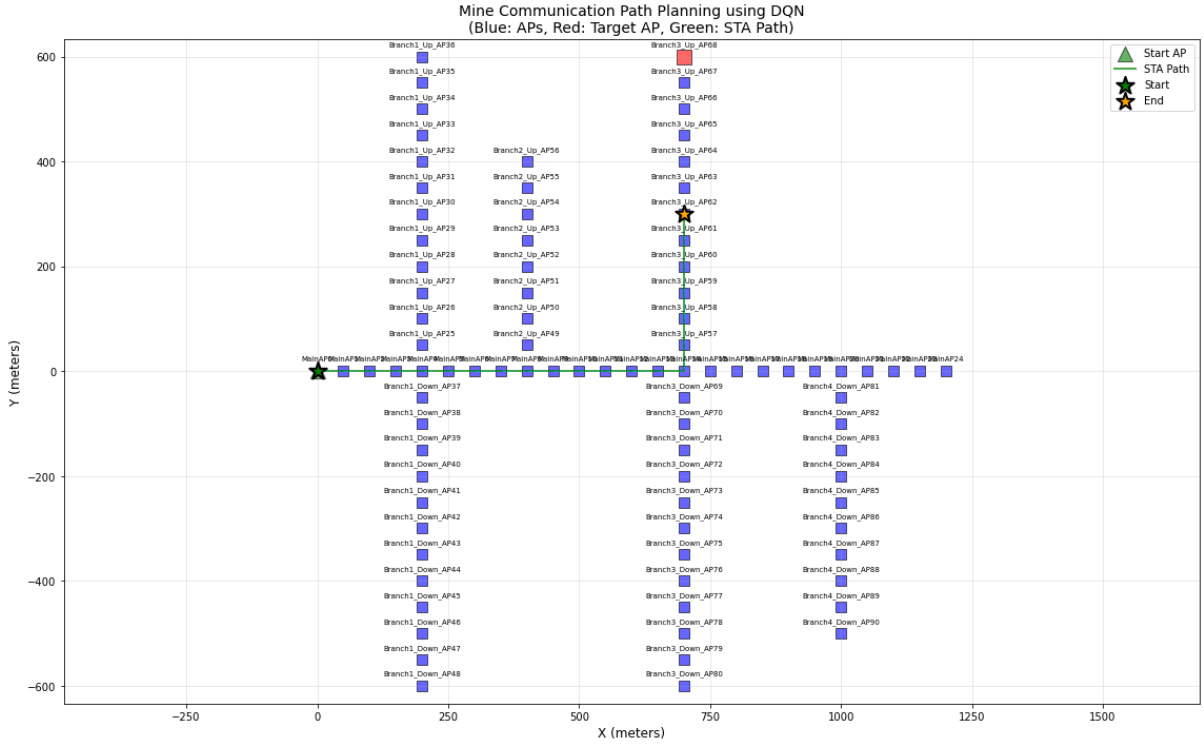

Figure 5.2: Result of optimal network route

branches. The green triangle at the left is the start AP, and the orange star in the upper branch is the target AP. The DQN agent first moves along the main corridor from the start towards the junction near the target branch. It then turns into the correct upper branch and climbs along that branch to reach the destination AP. Crucially, the path is composed of short, physically plausible hops between neighbouring APs inside the tunnel structure; there are no unrealistic "through-rock" shortcuts directly from the start to the target.

This demonstrates that the trained DQN has internalised both channel quality and tunnel geometry: it prefers routes that follow open corridors, avoid high-loss wall crossings, and still connect source and destination with a relatively small number of hops.

Chapter 6

Conclusion

In this work, we adopt a scenario-driven design for long-corridor, chain-topology networks with fixed nodes in GPS-denied environments, where routing and localization must rely on channel quality and distance cues rather than absolute positioning. The practical stack uses sub-GHz Wi-Fi HaLow to balance penetration, range and usable data rate, together with RSSI fingerprinting and a log-distance path loss model (optionally combined with trilateration) to provide positions that support routing decisions. The methodology follows a closed loop: we first measure the channel and calibrate the joint RSSI–distance–MCS relationship, then build an NS-3 "digital twin" of the corridor, and finally evaluate deep/RL-enhanced routing schemes against conventional baselines. Performance is assessed using both link-level metrics (RSSI, SNR, PER, RTT, MCS) and system-level metrics (latency, throughput, delivery ratio, routing overhead). Measurement-to-model calibration aligns NS-3 link behaviour with field observations around walls and corners, ensuring that a paper-shortest path is not mistaken for a truly usable route. On top of this calibrated model, we prototype a position-aware routing scheme with multi-factor link scoring (RSSI, load, stability, distance) to reduce flooding scope and path length, and we couple localization with routing by feeding RMSE-checked positions into route selection to improve delivery rate under corridor constraints. Looking ahead, we identify several extensions: MCS threshold mapping by expanding datasets across bandwidths, wall types and corners and fitting piecewise probabilistic RSSI-to-MCS curves; multi-objective reward design that combines connectivity, latency, energy and overhead while constraining node energy and congestion to avoid greedy-shortest behaviour; and robustness and safety enhancements through adversarial training with link and node outages and interference spikes, while tracking recovery time and service continuity.

Bibliography

[1] A. M. Damayanti and Khairunnisa, "Digital transformation in coal mining: Iot-based air quality monitoring system," in *2023 International Conference on Advanced Mechatronics, Intelligent Manufacture and Industrial Automation (ICAMIMIA)*. Surabaya, Indonesia: IEEE, 2023, pp. 1–4.

[2] G. Hui and J. Song, "Numerical calculation of mine pressure in deep immerged tunnel of protection layer mining," in *2011 International Conference on Remote Sensing, Environment and Transportation Engineering (RSETE)*. Nanjing, China: IEEE, 2011, pp. 5111–5114.

[3] T. Golubeva, Y. Zaitsev, S. Konshin, and I. Duisenbek, "A study on the wi-fi radio signal attenuation in various construction materials (obstacles)," in *2018 Tenth International Conference on Ubiquitous and Future Networks (ICUFN)*, 2018, pp. 718–723.

[4] J. Purat, N. J. Lehmann, M.-U. Karagülle, and A. Voisard, "Halownet - a wifi halow network-based information system for the provision of multi-sided applications for medical emergency scenarios," in *2022 IEEE 10th International Conference on Healthcare Informatics (ICHI)*. Rochester, MN, USA: IEEE, 2022, pp. 519–521.

[5] T. Porselvi, S. G. CS, J. B, P. K, and S. B. S, "Iot based coal mine safety and health monitoring system using lorawan," in *2021 3rd International Conference on Signal Processing and Communication (ICPSC)*, 2021, pp. 49–53.

[6] W. S. . F. F. Pandi, S., "Reliable low latency wireless mesh networks—from myth to reality," in *2018 15th IEEE Annual Consumer Communications & Networking Conference*, Las Vegas, USA, 2018, p. 1–2.

[7] Q. Zhu, X. Jiang, F. Zou *et al.*, "Improvement on multi-hop performance of underground mine emergency communication system based on wmn," *Journal of Computer Applications*, vol. 32, no. 3, pp. 800–803, 2012.

[8] A. Emslie, R. Lagace, and P. Strong, "Theory of the propagation of uhf radio waves in coal mine tunnels," *IEEE Transactions on Antennas and Propagation*, vol. 23, no. 2, pp. 192–205, 1975.

[9] E. Alotaibi and B. Mukherjee, "A survey on routing algorithms for wireless ad-hoc and mesh networks," *Computer Networks*, vol. 56, no. 2, pp. 940–965, 2012. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S138912861100377X

[10] S. Mittal and P. Kaur, "Performance comparison of aodv, dsr and zrp routing protocols in manets," in *International Conference on Advances in Computing, Control, and Telecommunication Technologies*. Bangalore, India: IEEE, 2009, pp. 165–168.

[11] S. Bari, F. Anwar, and M. H. Masud, "Performance study of hybrid wireless mesh protocol (hwmp) for ieee 802.11s wlan mesh networks," in *International Conference on Computer and Communication Engineering (ICCCE)*. Kuala Lumpur, Malaysia: IEEE, 2012, pp. 712–716.

[12] J. Gao, "Research on hwmp routing protocol based on mesh network," Master's thesis, Beijing Jiaotong University, Beijing, China, 2016.

[13] Q. Li, Z. Huo, L. Wen *et al.*, "Development of coal mine emergency rescue system based on wireless mesh network," *Industry and Mine Automation*, no. 6, pp. 39–43, 2012.

[14] S. Li and X. Huang, "Indoor fingerprint positioning based on wifi channel state information," in *2023 42nd Chinese Control Conference (CCC)*, 2023, pp. 3526–3531.

[15] R. Joseph and S. B. Sasi, "Indoor positioning using wifi fingerprint," in *2018 International Conference on Circuits and Systems in Digital Enterprise Technology (ICCSDET)*, 2018, pp. 1–3.

[16] H. Xujian and W. Hao, "Wifi indoor positioning algorithm based on improved kalman filtering," in *2016 International Conference on Intelligent Transportation, Big Data Smart City (ICITBS)*, 2016, pp. 349–352.

[17] J. Feng, Z. Liu, and Y. Ji, “Wireless channel loss analysis - a case study using wifi-direct,” in *2014 International Wireless Communications and Mobile Computing Conference (IWCMC)*, 2014, pp. 244–249.